

API Reference

Building an MQTT Server or Client with libterae

Emilio Schininà, M.Sc. in Automation Engineering
Version 0.2
November 8, 2026

Copyright © 2026 Emilio Schininà, M.Sc. in Automation Engineering. libterae is licensed under the GNU General Public License, version 3 or any later version, or under a commercial license.

Contents

1 Getting Started **1**
1.1 Two interfaces, two ways to run them 1
1.2 What holds for every call 2
1.3 Building a program. 3
1.4 Status codes 4
1.5 How a function is shown 5

2 The Server **7**
2.1 At a glance 7
2.2 The life of a Server 9
2.3 The configuration 9
2.4 The memory 13
2.5 Setting up 15
2.5.1 `mqtt_server_config_default` 15
2.5.2 `mqtt_server_meta_words` 16
2.5.3 `mqtt_server_init` 16
2.6 Running with `platform/` 17
2.6.1 `mqtt_net_server_run` 17
2.6.2 `mqtt_net_server_counters` 18
2.7 Running in a loop of the program's own 19
2.7.1 `mqtt_server_connection_open` 20
2.7.2 `mqtt_server_connection_closed` 21
2.7.3 `mqtt_server_receive_wanted` 21
2.7.4 `mqtt_server_receive_capacity` 22
2.7.5 `mqtt_server_receive_space` 23
2.7.6 `mqtt_server_receive_commit` 24
2.7.7 `mqtt_server_receive` 25
2.7.8 `mqtt_server_transmit_view` 26
2.7.9 `mqtt_server_transmit_consume` 26
2.7.10 `mqtt_server_should_close` 27

api-reference-en.tex Page iv November 8, 2026 13:41

2.7.11 mqtt_server_next_pending . . . 27
2.7.12 mqtt_server_tick . . . 28
2.7.13 mqtt_server_take_activity . . . 28
2.7.14 mqtt_server_accepted . . . 29
2.7.15 mqtt_server_keep_alive . . . 29
2.7.16 mqtt_server_idle . . . 30
2.7.17 mqtt_server_disconnect . . . 30
2.8 Messages from the program . . . 31
2.8.1 mqtt_server_publish . . . 31
2.8.2 mqtt_server_retained_next . . . 32
2.8.3 mqtt_server_retained_restore . . . 33
2.9 Keeping Session state across a restart . . . 33
2.9.1 mqtt_net_server_store_config . . . 34
2.9.2 mqtt_net_server_restore . . . 35
2.10 A store in a loop of the program's own . . . 35
2.10.1 mqtt_server_restore . . . 37
2.10.2 mqtt_server_store_commit . . . 38
2.10.3 mqtt_server_stored . . . 39
2.10.4 mqtt_server_store_failed . . . 39
2.10.5 mqtt_server_store_open . . . 40
2.10.6 mqtt_server_snapshot_due . . . 40
2.10.7 mqtt_server_snapshot_begin . . . 41
2.10.8 mqtt_server_snapshot_next . . . 42
2.10.9 mqtt_server_snapshot_done . . . 43
2.10.10 mqtt_server_snapshot_abort . . . 43
2.11 Checking a running Server . . . 43
2.11.1 mqtt_server_pool_audit . . . 44

3 The Client 45
3.1 At a glance . . . 45
3.2 The life of a Client . . . 47
3.3 The memory . . . 47
3.4 Events . . . 48
3.5 Setting up . . . 50
3.5.1 mqtt_client_init . . . 50
3.5.2 mqtt_client_assume_session . . . 50
3.6 Connecting . . . 51
3.6.1 mqtt_client_connect . . . 51
3.6.2 mqtt_client_auth . . . 52
3.6.3 mqtt_client_disconnect . . . 53
3.6.4 mqtt_client_should_close . . . 53
3.6.5 mqtt_client_connection_lost . . . 54

api-reference-en.tex Page v November 8, 2026 13:41

3.7 Messages and subscriptions 54
3.7.1 mqtt_client_publish 54
3.7.2 mqtt_client_subscribe 56
3.7.3 mqtt_client_unsubscribe 57
3.8 Running with platform/ 58
3.8.1 mqtt_net_client_open 58
3.8.2 mqtt_net_client_step 59
3.8.3 mqtt_net_client_close 60
3.9 Running in a loop of the program's own 60
3.9.1 mqtt_client_receive_capacity 61
3.9.2 mqtt_client_receive_space 61
3.9.3 mqtt_client_receive_commit 62
3.9.4 mqtt_client_receive 62
3.9.5 mqtt_client_transmit_view 63
3.9.6 mqtt_client_transmit_consume 63
3.9.7 mqtt_client_tick 64
3.10 Keeping Session State across a restart 64
3.10.1 mqtt_client_store_init 66
3.10.2 mqtt_net_client_store_config 66
3.10.3 mqtt_net_client_restore 67
3.10.4 mqtt_client_store_identifier 68
3.11 A store in a loop of the program's own 68
3.11.1 mqtt_client_restore 69
3.11.2 mqtt_client_store_commit 70
3.11.3 mqtt_client_stored 70
3.11.4 mqtt_client_store_failed 71
3.11.5 mqtt_client_store_open 71
3.11.6 mqtt_client_snapshot_due 72
3.11.7 mqtt_client_snapshot_begin 73
3.11.8 mqtt_client_snapshot_next 74
3.11.9 mqtt_client_snapshot_done 74
3.11.10 mqtt_client_snapshot_abort 75

4 Shared by Both **77**
4.1 Views of bytes 77
4.1.1 mqtt_utf8_from_text 78
4.2 Properties 78
4.2.1 mqtt5_properties_clear 80
4.2.2 mqtt5_properties_mark 80
4.2.3 mqtt5_properties_has 81
4.2.4 mqtt5_property_next 81

api-reference-en.tex Page vi November 8, 2026 13:41

4.3 Reason Codes 81
4.3.1 mqtt_reason_clear 82
4.3.2 mqtt_connect_clear. 82
4.3.3 mqtt5_reason_code 83
4.4 TLS 83
4.4.1 mqtt_tls_available. 83
4.5 The store 84
4.5.1 mqtt_store_fault_name 87
4.5.2 mqtt_platform_store_open. 88
4.5.3 mqtt_platform_store_close. 88
4.6 The system 90
4.6.1 mqtt_platform_time_ms 90
4.6.2 mqtt_platform_random_bytes 91
4.6.3 mqtt_platform_stop_requested 91
4.6.4 mqtt_platform_request_stop 92
4.7 The version of the library 93
4.7.1 mqtt_library_compatible 94
4.7.2 mqtt_library_version 95
4.7.3 mqtt_library_page_size 95

5 Examples **97**
5.1 Servers 97
5.2 Clients. 103

A Every Function **109**

1

Getting Started

This document lists every function a program calls to make an MQTT Server or an MQTT Client with libterae, what each one takes and gives back, and the structures a program fills in for them. The Server has Chapter 2 and the Client Chapter 3, so a program that makes only one of them reads only its chapter, with Chapter 4 for what both use. Chapter 5 has programs that build and run as they are.

The functions are described as they are at version 1.0.5. Each one's header has the same description in its own comment, which is the reference when the two disagree.

1.1 Two interfaces, two ways to run them

A Server is an `mqtt_server`, which `base_server.h` declares, and a Client is an `mqtt_client`, from `base_client.h`. Neither does any input or output of its own. They take the bytes that arrive, hand back the bytes to send, and are told the time, as Figure 1.1 shows. What moves the bytes between them and the network is a loop, and a program has two to choose from:

- **The loop of `platform/`.** `mqtt_net_server_run` (§2.6.1) runs a whole Server, and `mqtt_net_client_step` (§3.8.2) moves the bytes of a Client one step at a time. They do the sockets, TLS, WebSocket for a Server and the waiting, on the systems the library has a folder for. This is the short way, which the examples take.
- **A loop of the program's own.** The program reads and writes its own sockets, or a radio, or a serial line, and calls the Server or the Client with the bytes. A library built with `OS=none` has only this way. Sections 2.7 and 3.9 give the calls.

Table 1.1 says which headers a program includes for each.

api-reference-en.tex Page 2 November 8, 2026 13:41

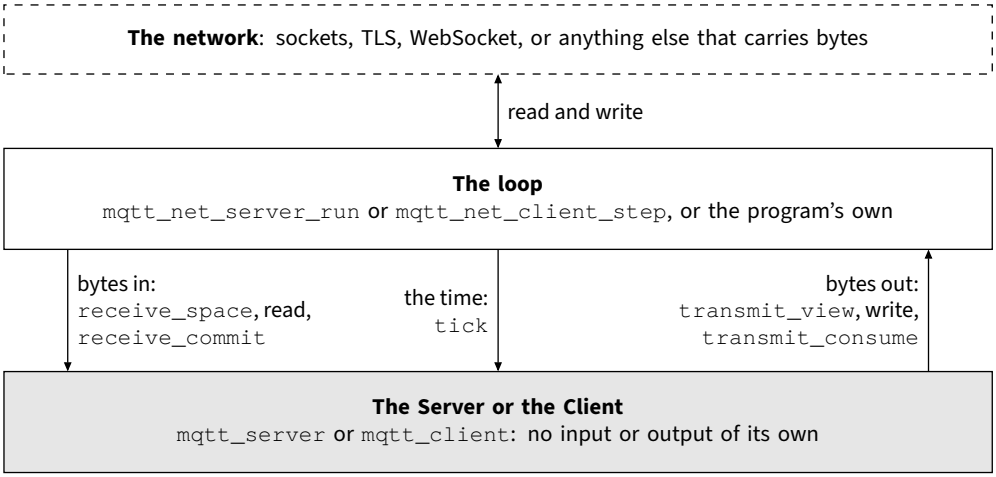


Figure 1.1 How bytes and time reach a Server or a Client. The names of the calls begin with `mqtt_server_` or `mqtt_client_`

Table 1.1 The headers a program includes

For	A Server	A Client
The interface	`base_server.h`	`base_client.h`
The loop of `platform/`	`base_net.h`	`base_net.h`
The clock, random bytes, stopping	`base_platform.h`	`base_platform.h`
Keeping Session state	`base_store.h`, and `base_platform.h` for a store of `platform/`	The same
Reason Codes	`v5_rc.h`, `v311_rc.h`, `v31_rc.h`	The same
Property Identifiers	`v5_prop.h`	`v5_prop.h`
Subscription Options	–	`v5_subscribe.h`

1.2 What holds for every call

- **The program gives all the memory.** A Server or a Client is given every array it will use when it is set up, in an `mqtt_server_storage` or an `mqtt_client_storage`, and never allocates. The arrays may be static, on a stack that lasts, or anywhere else, and must last as long as the Server or the Client.

api-reference-en.tex Page 3 November 8, 2026 13:41

They need not be cleared first. Their sizes are the limits: running out of anything is an ordinary outcome, reported as the protocol allows, and never a crash.

- **Treat the objects as opaque.** `mqtt_server` and `mqtt_client` are in the headers only so that the program can give their memory. Use the functions, not the fields.
- **One thread at a time.** Nothing locks. Calls on one Server or one Client must come one after the other. Different Servers and Clients have nothing in common, and can be used from different threads.
- **The time is an argument.** A call that needs the time takes it as `now_ms`, in milliseconds, from a clock that only goes forward, such as `mqtt_platform_time_ms` (§4.6.1). Protocol intervals keep the protocol's unit, seconds, as Keep Alive does.
- **Strings are views.** `mqtt_utf8`, `mqtt_binary` and `mqtt_payload` hold a length and a pointer to bytes they do not own, with no NUL at the end. What a call is given is copied if it is kept, and what an event hands out lasts only as long as the event (Section 4.1).
- **Outcomes are `mqtt_status`.** A function that can fail returns one of the values of Table 1.2. `MQTT_OK` means it did what it was asked.

1.3 Building a program

A program is compiled against `include/`, with the page size the library was built with, and linked with `libterae.a`:

```
cc -std=c99 -Iinclude -DMQTT_PAGE_SIZE=16384U server.c build/libterae.a -o server
```

`MQTT_PAGE_SIZE` sets the size of arrays the program gives the library, and the archive does not record it, so the two must agree. A build with TLS links the three archives of Mbed TLS after `libterae.a`, and a Windows program links `ws2_32` and `bcrypt`. A program can also take the library from the shared one `make shared` builds, which the system loads when the program starts, and then checks that it is one it can use (§4.7). *Building libterae* describes the builds.

api-reference-en.tex Page 4 November 8, 2026 13:41

1.4 Status codes

Table 1.2 The values of `mqtt_status`, from `base_status.h`

Value	Meaning
`MQTT_OK`	It did what it was asked.
`MQTT_INCOMPLETE`	Nothing yet: no whole item, nothing to read, or nothing taken.
`MQTT_MALFORMED_PACKET`	The bytes break the packet format, or a string is not valid UTF-8, which would make a packet that holds it malformed.
`MQTT_PROTOCOL_ERROR`	The packet is well formed but breaks the protocol.
`MQTT_UNSUPPORTED_PROTOCOL_VERSION`	The CONNECT asks for a version the library does not have.
`MQTT_CLIENT_IDENTIFIER_NOT_VALID`	The Server will not take that Client Identifier.
`MQTT_BAD_USER_NAME_OR_PASSWORD`	The User Name or Password was refused.
`MQTT_NOT_AUTHORIZED`	Refused by authorization.
`MQTT_BAD_AUTHENTICATION_METHOD`	The Server does not support that Authentication Method.
`MQTT_TOPIC_NAME_INVALID`	The Topic Name is not valid.
`MQTT_TOPIC_FILTER_INVALID`	The Topic Filter is not valid.
`MQTT_TOPIC_ALIAS_INVALID`	The Topic Alias is not valid.
`MQTT_PACKET_TOO_LARGE`	The packet is larger than its limit.
`MQTT_RECEIVE_MAXIMUM_EXCEEDED`	More QoS 1 and QoS 2 messages in flight than Receive Maximum allows.
`MQTT_PACKET_IDENTIFIER_IN_USE`	The Packet Identifier is in use.
`MQTT_PACKET_IDENTIFIER_NOT_FOUND`	The Packet Identifier is not in use.
`MQTT_PAYLOAD_FORMAT_INVALID`	The Payload does not match its Payload Format Indicator.
`MQTT_BUFFER_TOO_SMALL`	The result does not fit where it should go.
`MQTT_QUOTA_EXCEEDED`	One of the arrays the program gave is full.
`MQTT_INVALID_ARGUMENT`	An argument is out of range or `NULL`, or an object was not set up.

continues

api-reference-en.tex Page 5 November 8, 2026 13:41

Table 1.2 The values of `mqtt_status` (*continued*)

Value	Meaning
`MQTT_INVALID_STATE`	Not allowed now, as before the connection is accepted.
`MQTT_NOT_FOUND`	The item looked for does not exist.
`MQTT_CONNECTION_CLOSED`	The network connection has ended, or could not be made.
`MQTT_TLS_ERROR`	TLS failed, or the library was built without it.
`MQTT_RING_FULL`	No room for all the bytes, so none were added.
`MQTT_STORE_DAMAGED`	Stored state is damaged, or doesn't fit, so nothing of it was taken (Section 4.5).

1.5 How a function is shown

Each function has a subsection of its own: a sentence on what it does, its declaration as its header has it, with the header named at the foot of the box, and a table of its parameters and what it returns. Under a parameter's name is its direction: *in* for what the function reads, *out* for what it writes, *in, out* for both. For a pointer, it is the direction of what it points to. Every pointer must be valid, or `NULL` where its row says it may be. A `NULL` pointer anywhere else is refused as an object that isn't set up is: the call changes nothing, and returns `MQTT_INVALID_ARGUMENT`, 0, `false` or what it gives for nothing found.

```
uint32_t mqtt_server_tick(mqtt_server *server, uint64_t now_ms);
```

See include/base_server.h

Parameter	Meaning
`server` *in, out*	The server.
`now_ms` *in*	The time, in milliseconds.
Returns	How many milliseconds until it should be called again.

2

The Server

A Server takes Network Connections from Clients of MQTT 3.1, 3.1.1 and 5.0 on the same listener, and does everything the protocol asks of a Server: Sessions, subscriptions, retained messages, Will Messages, QoS 1 and 2, Topic Aliases, Shared Subscriptions and enhanced authentication. The program decides only how much memory it has, its configuration, and who may connect, publish and subscribe.

2.1 At a glance

Table 2.1 The functions of a Server

Function	What it does	§
Setting up		
`mqtt_server_config_default`	Fills a configuration with the defaults	2.5.1
`mqtt_server_meta_words`	Counts the page metadata a storage needs	2.5.2
`mqtt_server_init`	Sets up a server	2.5.3
Running with `platform/`		
`mqtt_net_server_run`	Runs the server until the program stops	2.6.1
`mqtt_net_server_counters`	Reads the counts of the network layer	2.6.2
Running in a loop of the program's own		
`mqtt_server_connection_open`	Reports a new connection	2.7.1
`mqtt_server_connection_closed`	Reports that a connection is gone	2.7.2
`mqtt_server_receive_wanted`	Says whether to read a connection now	2.7.3

continues

api-reference-en.tex Page 8 November 8, 2026 13:41

Table 2.1 The functions of a Server (*continued*)

Function	What it does	§
`mqtt_server_receive_capacity`	Says how many bytes it may be given	2.7.4
`mqtt_server_receive_space`	Gives the places to read bytes into	2.7.5
`mqtt_server_receive_commit`	Takes the bytes read into them	2.7.6
`mqtt_server_receive`	Takes bytes from a buffer of the program's	2.7.7
`mqtt_server_transmit_view`	Gives the bytes to send	2.7.8
`mqtt_server_transmit_consume`	Says how many of them went	2.7.9
`mqtt_server_should_close`	Says when to close the socket	2.7.10
`mqtt_server_next_pending`	Hands out connections with work to do	2.7.11
`mqtt_server_tick`	Runs the timers	2.7.12
`mqtt_server_take_activity`	Says what a connection has sent	2.7.13
`mqtt_server_accepted`	Says whether its CONNECT was accepted	2.7.14
`mqtt_server_keep_alive`	Gives the Keep Alive in force	2.7.15
`mqtt_server_idle`	Says whether a connection holds nothing	2.7.16
`mqtt_server_disconnect`	Closes a connection from the Server's side	2.7.17
Messages from the program		
`mqtt_server_publish`	Publishes a message from the Server's side	2.8.1
`mqtt_server_retained_next`	Hands out the retained messages, to save them	2.8.2
`mqtt_server_retained_restore`	Puts back a saved retained message	2.8.3
Keeping Session state		
`mqtt_net_server_store_config`	Sets a configuration to keep a store of `platform/`	2.9.1
`mqtt_net_server_restore`	Puts back what that store holds	2.9.2
`mqtt_server_restore`	Puts back what a store holds, on any medium	2.10.1
`mqtt_server_store_commit`	Closes the records of a round into a group	2.10.2
`mqtt_server_stored`	Says groups are stored, so their answers go	2.10.3

continues

api-reference-en.tex Page 9 November 8, 2026 13:41

Table 2.1 The functions of a Server (*continued*)

Function	What it does	§
`mqtt_server_store_failed`	Says records may have been lost	2.10.4
`mqtt_server_store_open`	Says whether records wait to be closed	2.10.5
`mqtt_server_snapshot_due`	Says whether a snapshot should begin	2.10.6
`mqtt_server_snapshot_begin`	Begins a snapshot	2.10.7
`mqtt_server_snapshot_next`	Gives its next record	2.10.8
`mqtt_server_snapshot_done`	Says it is whole in every copy	2.10.9
`mqtt_server_snapshot_abort`	Gives it up	2.10.10
Checking		
`mqtt_server_pool_audit`	Checks the pool of blocks	2.11.1

2.2 The life of a Server

1. Fill an `mqtt_server_config` with `mqtt_server_config_default` (§2.5.1), and change what the program needs: its limits, and the callbacks that decide who may connect, publish and subscribe (Section 2.3).
2. To keep its Sessions across a restart, open a store, and set the configuration to keep them there (Section 2.9).
3. Give the memory in an `mqtt_server_storage` (Section 2.4).
4. Set the server up with `mqtt_server_init` (§2.5.3), and put back what the store holds, if it has one.
5. Run it: with `mqtt_net_server_run` (§2.6.1), which returns when the program is asked to stop, or in a loop of the program's own (Section 2.7).

2.3 The configuration

`mqtt_server_config_default` (§2.5.1) writes every field of Table 2.2. A limit that a Property of MQTT 5.0 also names has that Property's meaning, and the Server tells Clients of 5.0 the ones that differ from the protocol's own defaults.

api-reference-en.tex Page 10 November 8, 2026 13:41

Table 2.2 The fields of `mqtt_server_config`

Field	Default	Meaning
Callbacks, each `NULL` for none		
`user_tag`	0	Handed to every callback, `store_append` too, to find the program's own data. `mqtt_net_server_store_config` (§2.9.1) sets it to the store.
`authenticate`	`NULL`: all connect	Decides whether a Client may connect.
`auth_step`	`NULL`: no method	Runs enhanced authentication, a step at a time.
`authorize_publish`	`NULL`: all publish	Decides whether a Client may publish to a topic.
`authorize_subscribe`	`NULL`: all subscribe	Decides whether a Client may subscribe to a Topic Filter.
`random_bytes`	`NULL`: numbered	Gives the random bytes of each Assigned Client Identifier. Numbered identifiers can be guessed, and a Client that guesses another's can take its Session, so give one.
Limits on what Clients send		
`maximum_packet_size`	268,435,455	The largest packet taken from a Client.
`maximum_connect_size`	1 MiB	The largest packet from a Client not yet accepted.
`receive_maximum`	65,535	The most QoS 2 messages a Client may have in progress, from 1 to 65,535.
`maximum_client_identifier`	256	The longest Client Identifier, at least 23.
`topic_alias_maximum`	16	Topic Aliases a Client may set, which also sizes `aliases` in the storage.
`maximum_qos`	2	Maximum QoS. A value above 2 is taken as 2.

continues

api-reference-en.tex Page 11 November 8, 2026 13:41

Table 2.2 The fields of `mqtt_server_config` (*continued*)

Field	Default	Meaning
Limits on what the Server keeps and sends		
`maximum_inflight`	1,024	QoS 1 and 2 messages sent to a Client and not yet acknowledged, from 1 to 65,535.
`maximum_queued`	4,096	Messages waiting to be sent to one Session.
`maximum_session_expiry`	never	The longest Session Expiry Interval, in seconds.
`persistent_session_expiry`	never	The Session Expiry Interval given to a Session of MQTT 3.1 and 3.1.1 that is not clean.
Time		
`connect_timeout_ms`	30,000	How long a new connection has to send its CONNECT and end any authentication. 0: no limit.
`retry_interval_ms`	30,000	How long before an MQTT 3.1 Client's unanswered messages are sent again, doubling each time. 0: only when a Session resumes.
`server_keep_alive`	0	Server Keep Alive, in seconds, or 0 to take the Client's.
Features		
`retain_available`	true	Retain Available.
`wildcard_subscription_available`	true	Wildcard Subscription Available.
`subscription_identifier_available`	true	Subscription Identifier Available.
`shared_subscription_available`	true	Shared Subscription Available.
`shared_subscription_311`	false	Treat `\$share/` as shared with MQTT 3.1.1 too, as other Servers do.
`dollar_publish_allowed`	false	Let Clients of 3.1.1 and 5.0 publish to topics that start with `$`.

continues

api-reference-en.tex Page 12 November 8, 2026 13:41

Table 2.2 The fields of `mqtt_server_config` (*continued*)

Field	Default	Meaning
`report_no_matching_subscribers`	false	Answer with Reason Code 0x10 when nobody is subscribed.
The pool of blocks, in blocks		
`pool_soft_quota`	4	What a connection may hold while the pool keeps only `pool_reserve` free.
`pool_hard_quota`	16	The most one connection may hold. About three quarters of it is the largest packet sent from its ring; a larger one is sent from the arena.
`pool_reserve`	64	Free blocks kept for replies and for connections that have just started.
`pool_borrow_reserve`	128	Free blocks kept back from a connection over its soft quota, at least `pool_reserve`.
The store		
`store_append`	`NULL`: keeps nothing	Hands each record of the journal to the program, to keep (Section 2.9).
`store_sessions`	`MQTT_STORE_MEMORY`	What the records of Sessions have to outlast: their subscriptions, Will Messages and lifetimes. `MQTT_STORE_MEMORY` writes nothing at all.
`store_messages`	`MQTT_STORE_MEMORY`	What the records of messages have to outlast: those waiting, in flight or retained, and the QoS 2 Packet Identifiers received. `MQTT_STORE_MEMORY` keeps the Sessions without their messages.
`store_snapshot_bytes`	64 MiB	Bytes of journal after which a snapshot is due, so that the journal starts again.

The callbacks run during the call that needs them, on the program's thread. Their

api-reference-en.tex Page 13 November 8, 2026 13:41

types are these:

```
typedef mqtt_status (*mqtt_server_authenticate)(uint32_t user_tag,
                                                const mqtt_connect *packet);
typedef mqtt_status (*mqtt_server_auth_step)(uint32_t user_tag,
                                             uint32_t connection,
                                             const mqtt_utf8 *method,
                                             const mqtt_binary *data_in,
                                             mqtt_binary *data_out);
typedef bool (*mqtt_server_authorize)(uint32_t user_tag,
                                      const mqtt_utf8 *client_identifier,
                                      const mqtt_utf8 *topic);
typedef mqtt_status (*mqtt_server_random_bytes)(uint32_t user_tag,
                                                uint8_t *data,
                                                uint32_t length);
```

See include/base_server.h

- **`authenticate`.** Is given the whole CONNECT packet, User Name and Password among it (Table 3.4), and returns `MQTT_OK`, `MQTT_BAD_USER_NAME_OR_PASSWORD` or `MQTT_NOT_AUTHORIZED`.
- **`auth_step`.** Is given the Authentication Method and the Authentication Data that came, and writes the Authentication Data to send into `data_out`, whose bytes must stay valid until it returns. It returns `MQTT_OK` when done, `MQTT_INCOMPLETE` to go on, or a failure. The connection's index lets it keep its own state for the exchange.
- **`authorize_publish`, `authorize_subscribe`.** Are given the Client Identifier and the Topic Name or Topic Filter, and return true if the Client may.
- **`random_bytes`.** Fills `length` bytes from a source an attacker can't predict, and returns `MQTT_OK`, or anything else if it has none. `mqtt_platform_random_bytes` (§4.6.2) is such a source.

2.4 The memory

An `mqtt_server_storage` names each array the server will use and how many entries it has. The counts are the server's capacity. Listing `server_setup.c` in Section 5.1 fills one in.

api-reference-en.tex Page 14 November 8, 2026 13:41

Table 2.3 The fields of `mqtt_server_storage`

Field	What to give
Connections	
`connections`, `connection_count`	One `mqtt_connection` for each connection open at once.
`aliases`	`connection_count` times `topic_alias_maximum` entries of `mqtt_topic_alias`, or `NULL` if that maximum is 0.
Bytes in transit	
`pool_bytes`, `pool_next`	`pool_block_count` times `pool_block_size` bytes, and one `uint32_t` for each block. Every connection's bytes to send and received bytes come from here.
`pool_block_size`	Bytes in a block, at most 4,096. Times `pool_hard_quota`, at least 1,024.
`pool_block_count`	More than `pool_borrow_reserve` and `pool_hard_quota` together.
`receive_scratch`, `receive_scratch_size`	At least 16 bytes, where a packet that arrived in several blocks is gathered. Its size is the largest packet kept in blocks: larger ones go to the arena.
Sessions and messages	
`sessions`, `session_count`	One `mqtt_session` for each Session kept, connected or not.
`session_table`, `session_table_count`	At least `session_count` entries, to find a Session by its Client Identifier.
`deliveries`, `delivery_count`	One `mqtt_delivery` for each message on its way to one Session.
`messages`, `message_count`	One `mqtt_message` for each message stored, however many Sessions it goes to.
`subscriptions`, `subscription_count`	One `mqtt_subscription` for each subscription of each Session.
`groups`, `group_count`	One `mqtt_share_group` for each Shared Subscription, or `NULL` and 0 for none.
Topics	
`nodes`, `node_count`	One `mqtt_tree_node` for each level of every topic subscribed to or retained, at most 65,535.
`branches`, `branch_count`	At least `node_count` − 2 entries of `mqtt_tree_branch`.

continues

api-reference-en.tex Page 15 November 8, 2026 13:41

Table 2.3 The fields of `mqtt_server_storage` (*continued*)

Field	What to give
`topic_text`, `topic_text_pages`	Pages of `MQTT_PAGE_SIZE` bytes for the text of the levels.
Everything of varying size	
`arena_bytes`, `arena_pages`	Pages of `MQTT_PAGE_SIZE` bytes for messages, Client Identifiers and the packets being gathered.
`page_meta`, `page_meta_words`	At least as many words as `mqtt_server_meta_words` (§2.5.2) gives for the rest of the storage.

Pages that are never touched cost nothing on a system that maps memory as it is used, so the arena can be given generously.

2.5 Setting up

2.5.1 `mqtt_server_config_default`

Fills a configuration with the defaults of Table 2.2. Every field is written, so the memory may hold anything before.

```
void mqtt_server_config_default(mqtt_server_config *config);
```

See include/base_server.h

Parameter	Meaning
`config` *out*	The configuration.
Returns	Nothing.

api-reference-en.tex Page 16 November 8, 2026 13:41

2.5.2 `mqtt_server_meta_words`

Counts the words of page metadata a storage needs: `MQTT_PAGE_META_WORDS` for each page of every arena. Fill in every other field of the storage first. A program that sizes `page_meta` when it is compiled can count the pages itself, as `server_setup.c` does.

```
uint32_t mqtt_server_meta_words(const mqtt_server_storage *storage);
```

See include/base_server.h

Parameter	Meaning
`storage` *in*	The storage, with its counts and numbers of pages set.
Returns	The number of words for `page_meta`, or `0xFFFFFFFF` if no array could hold them.

2.5.3 `mqtt_server_init`

Sets up a server. The configuration is copied, and so is the description of the memory: the arrays themselves are the server's from now on.

```
mqtt_status mqtt_server_init(mqtt_server *server,
                             const mqtt_server_config *config,
                             const mqtt_server_storage *storage);
```

See include/base_server.h

Parameter	Meaning
`server` *out*	The server.
`config` *in*	The configuration.
`storage` *in*	The memory.
Returns	`MQTT_OK`, or `MQTT_INVALID_ARGUMENT` if the configuration or the storage breaks a rule of Tables 2.2 and 2.3.

Every other function refuses a server that this has not set up.

api-reference-en.tex Page 17 November 8, 2026 13:41

2.6 Running with `platform/`

2.6.1 `mqtt_net_server_run`

Runs a server until the program is asked to stop. It listens on the ports the options name, accepts connections, waits for the sockets that are ready in tiers, so that each wait looks at few, moves their bytes in and out, does TLS and WebSocket, and runs the timers, all on the calling thread.

```
mqtt_status mqtt_net_server_run(mqtt_server *server,
                                mqtt_net_connection *connections,
                                const mqtt_net_server_options *options);

                                                    See include/base_net.h
```

Parameter	Meaning
`server` *in, out*	A server that `mqtt_server_init` (§2.5.3) has set up.
`connections` *out*	One `mqtt_net_connection` for each connection of the server: `connection_count` entries.
`options` *in*	Where to listen, and with what (Table 2.4).
Returns	`MQTT_OK` once it has stopped, or why it could not start.

It stops when `mqtt_platform_stop_requested` (§4.6.3) says so: on a POSIX system, after `SIGINT` or `SIGTERM`, which it starts to watch, and on any system after `mqtt_platform_request_stop` (§4.6.4). It then closes every connection and returns. A TLS port needs a library built with TLS, and loads the certificate and key itself.

With a store in its options, it first writes a whole snapshot, before it listens. Each round then closes the records the round wrote into a group, syncs it if its COMMIT asks for that, lets the answers that waited for it go, and writes some of the next snapshot when one is due. When it stops, it closes and syncs the last records, and leaves the store for the program to close. A store given to a server that keeps none, or none given to one that keeps one, is `MQTT_INVALID_ARGUMENT`.

api-reference-en.tex Page 18 November 8, 2026 13:41

Table 2.4 The fields of `mqtt_net_server_options`

Field	Meaning
`host`	The local address to bind, or `NULL` for all.
`port`	The port for MQTT over TCP, as text, or `NULL` for none.
`tls_port`	The port for MQTT over TLS, or `NULL`.
`websocket_port`	The port for MQTT over WebSocket, or `NULL`.
`certificate_file`	The Server's certificate chain, in PEM form, for TLS.
`key_file`	The Server's private key, in PEM form.
`key_password`	The key's password, or `NULL` if it has none.
`ca_file`	Certificates to require a client certificate to chain to, or `NULL` not to ask for one.
`tier`	How the tiers behave, or `NULL` for the defaults.
`send_buffer`	Bytes of the system's send buffer for each connection, or 0 to leave it to the system.
`store`	The store the server keeps its Session state in, from `mqtt_platform_store_open` (§4.5.2), or `MQTT_STORE_NONE`, which `base_platform.h` defines, to keep none. Nothing fills in the options, so this must be set too.

2.6.2 `mqtt_net_server_counters`

Reads what the network layer of the server that runs, or ran last, has counted: in `tier`, the tiers' counts, an `mqtt_tier_counters`; in `batches`, how often bytes were found waiting to be sent; and in `write_calls`, how many calls wrote to a socket or to TLS.

```
void mqtt_net_server_counters(mqtt_net_counters *counters);
```

See include/base_net.h

Parameter	Meaning
`counters` *out*	The counts.
Returns	Nothing.

api-reference-en.tex Page 19 November 8, 2026 13:41

2.7 Running in a loop of the program's own

A program that moves the bytes itself gives each connection an index from `mqtt_server_connection_open` (§2.7.1), and keeps its socket beside it. A round of its loop does this:

1. Call `mqtt_server_tick` (§2.7.12), which runs the timers and says how long the loop may wait.
2. Wait for the sockets that are ready, no longer than that.
3. For a new connection, call `mqtt_server_connection_open` (§2.7.1). With no room, close the socket.
4. For a connection that can be read, and while `mqtt_server_receive_wanted` (§2.7.3) says so, read into the places of `mqtt_server_receive_space` (§2.7.5), then call `mqtt_server_receive_commit` (§2.7.6) with what was read, even nothing.
5. For a connection that has bytes to send, write the runs of `mqtt_server_transmit_view` (§2.7.8), then call `mqtt_server_transmit_consume` (§2.7.9) with what the socket took.
6. With a store, close the round's records with `mqtt_server_store_commit` (§2.10.2), keep them, and report them with `mqtt_server_stored` (§2.10.3), so that the answers that waited for them can go (Section 2.10).
7. Call `mqtt_server_next_pending` (§2.7.11) until it gives `MQTT_SERVER_NONE`, and do steps 4 and 5 for each connection it gives: messages for a connection come from the others, and its answers from the store, so it can have bytes to send, or room to read again, while its own socket is not ready.
8. When `mqtt_server_should_close` (§2.7.10) says so, or the socket fails, close the socket and call `mqtt_server_connection_closed` (§2.7.2).

Listing `server_loop.c` in Section 5.1 is such a loop, kept short by looking at every socket in every round.

api-reference-en.tex Page 20 November 8, 2026 13:41

2.7.1 `mqtt_server_connection_open`

Reports a new Network Connection, and gives it an index, which every later call about it takes.

```
mqtt_status mqtt_server_connection_open(mqtt_server *server, uint64_t now_ms,
                                        uint32_t *connection);

                                                    See include/base_server.h
```

Parameter	Meaning
`server` *in, out*	The server.
`now_ms` *in*	The time.
`connection` *out*	Its index, less than `connection_count`.
Returns	`MQTT_OK`, or `MQTT_QUOTA_EXCEEDED` if every connection is in use: close the socket.

The connection has `connect_timeout_ms` to send its CONNECT, or for ever if that is 0.

api-reference-en.tex Page 21 November 8, 2026 13:41

2.7.2 mqtt_server_connection_closed

Reports that a Network Connection is gone: the Client closed it, the socket failed, or `mqtt_server_should_close` (§2.7.10) said to close it. The Server then does what the protocol asks when a connection ends, and the index can no longer be used.

```
void mqtt_server_connection_closed(mqtt_server *server, uint32_t connection,
                                   uint64_t now_ms);
```

See include/base_server.h

Parameter	Meaning
`server` *in, out*	The server.
`connection` *in*	Its index. One that is not open is ignored.
`now_ms` *in*	The time.
Returns	Nothing.

2.7.3 mqtt_server_receive_wanted

Decides whether to read from a connection now. With no room, a connection holding part of a packet whose size is known moves it to the arena, so that its blocks go back to the pool. One with nothing to send waits for blocks, and `mqtt_server_next_pending` (§2.7.11) hands it out when some come back: ask again then.

```
bool mqtt_server_receive_wanted(mqtt_server *server, uint32_t connection);
```

See include/base_server.h

Parameter	Meaning
`server` *in, out*	The server.
`connection` *in*	The connection.
Returns	true if it has room for bytes.

A connection that is not read leaves its bytes in the system's buffer, which fills, and the Client's sending slows: this is how a Server short of memory holds its Clients

api-reference-en.tex Page 22 November 8, 2026 13:41

back.

2.7.4 `mqtt_server_receive_capacity`

Finds how many bytes a connection may be given now. Reading no more than this keeps room for the replies, which never take more than the packets they answer.

```
uint32_t mqtt_server_receive_capacity(const mqtt_server *server,
                                      uint32_t connection);

                                                    See include/base_server.h
```

Parameter	Meaning
`server` *in*	The server.
`connection` *in*	The connection.
Returns	The number of bytes: zero while the Client is not reading what it is sent, while the pool has too few blocks free, or for a connection that is not open.

api-reference-en.tex Page 23 November 8, 2026 13:41

2.7.5 mqtt_server_receive_space

Finds where the next bytes a connection sends can go, and takes the blocks for them. Read from the socket into the places in order, as one `readv` does, then call `mqtt_server_receive_commit` (§2.7.6).

```
uint32_t mqtt_server_receive_space(mqtt_server *server, uint32_t connection,
                                   mqtt_space *spaces, uint32_t max_spaces);

                                                        See include/base_server.h
```

Parameter	Meaning
`server` *in, out*	The server.
`connection` *in*	The connection.
`spaces` *out*	The places, in the order bytes must fill them. They hold no more than `mqtt_server_receive_capacity` (§2.7.4).
`max_spaces` *in*	The number of entries `spaces` can hold.
Returns	The number of places, or zero if there is no room now.

Each call must be followed by `mqtt_server_receive_commit` (§2.7.6), with zero if nothing came, which gives back the blocks nothing was written to.

api-reference-en.tex Page 24 November 8, 2026 13:41

2.7.6 `mqtt_server_receive_commit`

Takes the bytes written into the places `mqtt_server_receive_space` (§2.7.5) gave, cuts them into packets, and acts on every whole one: the replies and the messages it sends to others are ready when it returns.

```
mqtt_status mqtt_server_receive_commit(mqtt_server *server, uint32_t connection,
                                       uint32_t count, uint64_t now_ms);

                                                        See include/base_server.h
```

Parameter	Meaning
`server` *in, out*	The server.
`connection` *in*	The connection.
`count` *in*	The number of bytes written, in order, which may be zero. No more than the places hold are counted.
`now_ms` *in*	The time.
Returns	`MQTT_OK`; `MQTT_INVALID_ARGUMENT` for a connection that is not open; or the reason the connection is now closing.

A connection that is closing still has bytes to send, such as a CONNACK or a DISCONNECT that says why: keep sending until `mqtt_server_should_close` (§2.7.10) says to close.

api-reference-en.tex Page 25 November 8, 2026 13:41

2.7.7 mqtt_server_receive

Takes bytes a connection has sent from a buffer of the program's, by way of `mqtt_server_receive_space` (§2.7.5) and `mqtt_server_receive_commit` (§2.7.6). It copies once more than they do, and suits a program that gets its bytes already in a buffer.

```
mqtt_status mqtt_server_receive(mqtt_server *server, uint32_t connection,
                                const uint8_t *data, uint32_t length,
                                uint64_t now_ms);

                                                        See include/base_server.h
```

Parameter	Meaning
`server` *in, out*	The server.
`connection` *in*	The connection.
`data` *in*	The bytes.
`length` *in*	The number of bytes in `data`.
`now_ms` *in*	The time.
Returns	`MQTT_OK`; `MQTT_INVALID_ARGUMENT` for a connection that is not open; `MQTT_RING_FULL` if there was no room for all the bytes, when the rest were not taken; or the reason the connection is now closing.

api-reference-en.tex Page 26 November 8, 2026 13:41

2.7.8 `mqtt_server_transmit_view`

Finds the next bytes to send on a connection, as runs that one `writev` can send together: one for each block of its ring, and one for a Payload sent straight from the arena.

```
uint32_t mqtt_server_transmit_view(mqtt_server *server, uint32_t connection,
                                   mqtt_run *runs, uint32_t max_runs);
```

See include/base_server.h

Parameter	Meaning
`server` *in, out*	The server.
`connection` *in*	The connection.
`runs` *out*	The runs, in the order they go. They stay valid until the next call about this server.
`max_runs` *in*	The number of entries `runs` can hold. Fewer than there are gives the first ones.
Returns	The number of runs, or zero if there is nothing to send now or the connection is not open.

2.7.9 `mqtt_server_transmit_consume`

Reports how many bytes of the view have been sent, from its first run on.

```
void mqtt_server_transmit_consume(mqtt_server *server, uint32_t connection,
                                  uint32_t count);
```

See include/base_server.h

Parameter	Meaning
`server` *in, out*	The server.
`connection` *in*	The connection. One that is not open is ignored.
`count` *in*	The number of bytes sent, which may end in any run. No more than are waiting are taken.
Returns	Nothing.

api-reference-en.tex Page 27 November 8, 2026 13:41

2.7.10 mqtt_server_should_close

Finds whether a connection should now be closed.

```
bool mqtt_server_should_close(const mqtt_server *server, uint32_t connection);
```

See include/base_server.h

Parameter	Meaning
`server` *in*	The server.
`connection` *in*	The connection.
Returns	true if it is closing and has nothing left to send, and false if it is not open.

Then close the socket, and call `mqtt_server_connection_closed` (§2.7.2).

2.7.11 mqtt_server_next_pending

Hands out, one at a time, the connections that have work to do since the last call: bytes to send that came from another connection, or room to read again after waiting for blocks.

```
uint32_t mqtt_server_next_pending(mqtt_server *server);
```

See include/base_server.h

Parameter	Meaning
`server` *in, out*	The server.
Returns	A connection, or `MQTT_SERVER_NONE` if there are no more.

api-reference-en.tex Page 28 November 8, 2026 13:41

2.7.12 `mqtt_server_tick`

Runs the timers: Keep Alive, the wait for a CONNECT, the Will Delay Interval, the Session Expiry Interval, message expiry, and the retries of MQTT 3.1.

```
uint32_t mqtt_server_tick(mqtt_server *server, uint64_t now_ms);

                                                        See include/base_server.h
```

Parameter	Meaning
`server` *in, out*	The server.
`now_ms` *in*	The time.
Returns	The number of milliseconds until it should run again, which is the longest the loop should wait.

2.7.13 `mqtt_server_take_activity`

Finds what a connection has sent since the last call, and starts again. A packet counts from its first byte. A network layer uses it to tell a Client that only keeps its connection alive from one with traffic, and wait for the first less often.

```
mqtt_server_activity mqtt_server_take_activity(mqtt_server *server,
                                               uint32_t connection);

                                                        See include/base_server.h
```

Parameter	Meaning
`server` *in, out*	The server.
`connection` *in*	The connection.
Returns	`MQTT_SERVER_ACTIVITY_NONE`, `MQTT_SERVER_ACTIVITY_PINGREQ` if only PINGREQ packets started, or `MQTT_SERVER_ACTIVITY_OTHER`. It is `MQTT_SERVER_ACTIVITY_NONE` for a connection that is not open.

api-reference-en.tex Page 29 November 8, 2026 13:41

2.7.14 mqtt_server_accepted

Finds whether a connection's CONNECT has been accepted.

```
bool mqtt_server_accepted(const mqtt_server *server, uint32_t connection);

                                                          See include/base_server.h
```

Parameter	Meaning
`server` *in*	The server.
`connection` *in*	The connection.
Returns	true from the CONNACK that accepted it until it starts to close.

2.7.15 mqtt_server_keep_alive

Finds the Keep Alive in force on a connection: the Client's, or the Server Keep Alive that replaced it.

```
uint16_t mqtt_server_keep_alive(const mqtt_server *server,
                                uint32_t connection);

                                                          See include/base_server.h
```

Parameter	Meaning
`server` *in*	The server.
`connection` *in*	The connection.
Returns	Seconds, or 0 for none or for a connection not accepted.

api-reference-en.tex Page 30 November 8, 2026 13:41

2.7.16 `mqtt_server_idle`

Finds whether a connection holds nothing: no part of a packet that has arrived, and nothing to send.

```
bool mqtt_server_idle(const mqtt_server *server, uint32_t connection);

                                                        See include/base_server.h
```

Parameter	Meaning
`server` *in*	The server.
`connection` *in*	The connection.
Returns	true if it holds nothing, and false if it is not open.

2.7.17 `mqtt_server_disconnect`

Closes a connection from the Server's side, telling the Client why where the protocol has a way to: a DISCONNECT with the Reason Code in MQTT 5.0. Keep sending until `mqtt_server_should_close` (§2.7.10) says to close.

```
void mqtt_server_disconnect(mqtt_server *server, uint32_t connection,
                            uint8_t reason_code);

                                                        See include/base_server.h
```

Parameter	Meaning
`server` *in, out*	The server.
`connection` *in*	The connection. One that is not open is ignored.
`reason_code` *in*	The Reason Code, from `v5_rc.h`, as `MQTT5_RC_ADMINISTRATIVE_ACTION`.
Returns	Nothing.

api-reference-en.tex Page 31 November 8, 2026 13:41

2.8 Messages from the program

2.8.1 `mqtt_server_publish`

Publishes an Application Message from the Server's side, to every Session subscribed to its topic, as if a Client had published it. The bytes are copied where they are kept, so they can be reused when it returns.

```
mqtt_status mqtt_server_publish(mqtt_server *server,
                                const mqtt_utf8 *topic_name,
                                const mqtt_payload *payload, uint8_t qos,
                                bool retain, uint64_t now_ms);
```

See include/base_server.h

Parameter	Meaning
`server` *in, out*	The server.
`topic_name` *in*	The Topic Name, which must be valid and hold no wildcard.
`payload` *in*	The Payload.
`qos` *in*	The QoS: 0, 1 or 2. No PUBLISH packet may have 3.
`retain` *in*	true to make it the retained message of its topic. With a Payload of zero bytes it removes that message instead, and is still sent to those subscribed.
`now_ms` *in*	The time.
Returns	`MQTT_OK`; `MQTT_TOPIC_NAME_INVALID`; `MQTT_MALFORMED_PACKET` if the Topic Name isn't valid UTF-8; `MQTT_INVALID_ARGUMENT` if the server isn't set up, `qos` is above 2, or it asks to retain and `retain_available` is false; or `MQTT_QUOTA_EXCEEDED`, below.

`MQTT_QUOTA_EXCEEDED` has three causes, and only the first leaves nothing done. If the Topic Name doesn't fit in a connection's ring, nothing was sent. If there was no room to retain the message, or to queue it for some Session, the Sessions that had room have it, so publishing it again sends them a second copy. A retained message that found no room still replaces the old one, which is gone.

api-reference-en.tex Page 32 November 8, 2026 13:41

2.8.2 `mqtt_server_retained_next`

Hands out the retained messages one at a time, for a program that saves them itself to outlast a restart. A server with a store keeps them there, with its Sessions (Section 2.9). Start with the cursor at zero.

```
bool mqtt_server_retained_next(const mqtt_server *server, uint64_t now_ms,
                               uint32_t *cursor, mqtt_server_retained *message);

                                                          See include/base_server.h
```

Parameter	Meaning
`server` *in*	The server.
`now_ms` *in*	The time, for what is left of each message's expiry.
`cursor` *in, out*	Where to look from: zero to start.
`message` *out*	The message (Table 2.5).
Returns	true if there was one, false if there are no more.

What it hands out, and the cursor, are only good until the server next runs: save them before calling anything else.

Table 2.5 The fields of `mqtt_server_retained`

Field	Meaning
`topic_name`	The Topic Name.
`properties`	Its Properties, encoded, without their length.
`payload`	The Payload.
`expiry_left_ms`	What is left of its Message Expiry Interval, or 0 if it has none.
`qos`	The QoS it was published with.

api-reference-en.tex Page 33 November 8, 2026 13:41

2.8.3 mqtt_server_retained_restore

Puts back a retained message that `mqtt_server_retained_next` (§2.8.2) handed out, after `mqtt_server_init` (§2.5.3) and before any Client connects.

```
mqtt_status mqtt_server_retained_restore(mqtt_server *server,
                                         const mqtt_server_retained *message,
                                         uint64_t now_ms);

                                                    See include/base_server.h
```

Parameter	Meaning
`server` *in, out*	The server.
`message` *in*	The message, as it was saved.
`now_ms` *in*	The time, from which its expiry runs again.
Returns	`MQTT_OK`; `MQTT_TOPIC_NAME_INVALID`; `MQTT_MALFORMED_PACKET` if the Topic Name isn't valid UTF-8; `MQTT_INVALID_ARGUMENT` if the server isn't set up, `retain_available` is false, the Payload is empty, the QoS is above 2, or the Properties aren't ones a PUBLISH can carry, or hold a Topic Alias, a Subscription Identifier or a Message Expiry Interval; or `MQTT_QUOTA_EXCEEDED`.

2.9 Keeping Session state across a restart

A server can keep in a store what it would otherwise lose when it stops: its Sessions, with their subscriptions and Will Messages, the messages waiting for them and in flight, the retained messages, and the Packet Identifiers of the QoS 2 messages it has received. Started again on the same store, it goes on from there, and a Client that resumes its Session finds it as it was. `server_store.c` in Section 5.1 is such a server.

- **A journal and a snapshot.** Each change is a record, handed to `store_append` when it is made. A snapshot, the whole state as records, lets the journal start again. It is kept in two copies or more, written one after the other, so that one is whole whatever happens to the other (Section 4.5).
- **Nothing acknowledged before it is kept.** The records of a round are

api-reference-en.tex Page 34 November 8, 2026 13:41

closed into a group. An answer that tells a Client something was kept waits until the program says its group is stored: CONNACK, PUBACK, PUBREC, PUBREL, PUBCOMP, SUBACK, UNSUBACK, and a QoS 2 PUBLISH sent for the first time. It waits in the connection's ring, so the bytes before it still go.

- **What each kind outlasts.** `store_sessions` and `store_messages` each take a value of Table 4.4: a loss of power, a crash of the process, or nothing. Any other value makes `mqtt_server_init` (§2.5.3) refuse the configuration.
- **All or nothing.** A restore checks every record as it checks what a Client sends. A store that is damaged, written by a later version, or more than this configuration and this memory can hold, is not taken at all: the server starts empty, and the store is set aside, never deleted. A Session never comes back by halves.

With `platform/`, a program opens a store with `mqtt_platform_store_open` (§4.5.2), sets the configuration with `mqtt_net_server_store_config` (§2.9.1) before `mqtt_server_init` (§2.5.3), puts back what the store holds with `mqtt_net_server_restore` (§2.9.2), and gives the store to `mqtt_net_server_run` (§2.6.1) in the `store` of its options. The loop does the rest, and the program closes the store with `mqtt_platform_store_close` (§4.5.3) when the loop has returned. A program with a loop of its own uses the calls of Section 2.10 instead.

2.9.1 `mqtt_net_server_store_config`

Sets a configuration to keep the server's Session state in a store of `platform/`. Records go to `mqtt_platform_store_append`, which is given the store as its tag, so `user_tag` becomes the store and every callback is given it. Sessions and messages are kept to outlast a loss of power, which the program may lower to `MQTT_STORE_WRITTEN` after this call. A snapshot is due after half a place of the store at most, so that a journal segment never fills its place.

```
void mqtt_net_server_store_config(mqtt_server_config *config, int32_t store);
```

See include/base_net.h

Parameter	Meaning
`config` *in, out*	The configuration, after `mqtt_server_config_default` (§2.5.1) and before `mqtt_server_init` (§2.5.3).
`store` *in*	The store, from `mqtt_platform_store_open` (§4.5.2).
Returns	Nothing.

api-reference-en.tex Page 35 November 8, 2026 13:41

2.9.2 mqtt_net_server_restore

Puts back what a store holds into a server just set up, before `mqtt_net_server_run` (§2.6.1). A store that is damaged, from a later version, or more than this server can take, is set aside untouched, a line on the console says why and where it went, and the server starts empty.

```
mqtt_status mqtt_net_server_restore(mqtt_server *server, int32_t store,
                                    mqtt_server_restore_entry *map,
                                    uint32_t map_count,
                                    mqtt_store_fault *fault);

                                                    See include/base_net.h
```

Parameter	Meaning
`server` *in, out*	The server, set up with the configuration of `mqtt_net_server_store_config` (§2.9.1), and holding nothing.
`store` *in*	The store.
`map` *out*	Room for the table that finds messages and deliveries by their numbers in the store: the storage's `message_count` and `delivery_count` entries together always suffice.
`map_count` *in*	The number of entries in `map`.
`fault` *out*	What was wrong with a store set aside (Table 4.7), or `MQTT_STORE_FAULT_NONE`.
Returns	`MQTT_OK` if the server can run, restored or empty; `MQTT_INVALID_ARGUMENT` or `MQTT_INVALID_STATE` as `mqtt_server_restore` (§2.10.1) gives them; or `MQTT_INVALID_STATE` if a damaged store could not be set aside, when the server must not run on it.

`MQTT_OK` comes back after a store was set aside too, so only `fault` tells the two apart. The reads go through a static buffer of `MQTT_SERVER_RESTORE_SPACE` bytes, about 128 KiB, of `platform/`'s own.

2.10 A store in a loop of the program's own

A program that keeps the store itself, on any medium, sets `store_append` and the two durabilities in the configuration, and `user_tag` for its own use. The records are its to keep, in the parts and places of Section 4.5: the copies of the snapshot, and two

api-reference-en.tex Page 36 November 8, 2026 13:41

places of the journal, one for each generation in turn. Then:

1. After `mqtt_server_init` (§2.5.3), put back what the store holds with `mqtt_server_restore` (§2.10.1). If it gives `MQTT_STORE_DAMAGED`, set the store aside, and start it again empty.
2. Before the first connection, take a whole snapshot: call `mqtt_server_snapshot_begin` (§2.10.7), start the first copy with the snapshot's header, add each record of `mqtt_server_snapshot_next` (§2.10.8) until `MQTT_NOT_FOUND`, and sync it. If `new_segment` is true, empty the journal place of the new generation, write the journal's header there, and sync it. Write the other copies, then call `mqtt_server_snapshot_done` (§2.10.9).
3. In each round, after the events, close the round's records with `mqtt_server_store_commit` (§2.10.2), sync if the COMMIT asks for that, and report them with `mqtt_server_stored` (§2.10.3), or call `mqtt_server_store_failed` (§2.10.4) if the sync failed. The connections whose answers can go now come out of `mqtt_server_next_pending` (§2.7.11).
4. When `mqtt_server_snapshot_due` (§2.10.6) says so, begin a snapshot, and start its journal segment at once, before the server runs again. Then write some of its records each round, stopping where `between` is true. The server runs in between, and what changes meanwhile goes to the new segment.
5. Don't wait for the sockets while `mqtt_server_store_open` (§2.10.5) says records are open: close them first.

api-reference-en.tex Page 37 November 8, 2026 13:41

2.10.1 mqtt_server_restore

Puts back what a store holds, into a server just set up. Every copy of the snapshot and both journal places are read and checked first. Then the newest copy that its journal segments follow on from is applied, and those segments up to their last COMMIT. Each record is checked as what a Client sends is, and has to fit this configuration.

```
mqtt_status mqtt_server_restore(mqtt_server *server,
                                const mqtt_store_source *source,
                                mqtt_server_restore_entry *map,
                                uint32_t map_count, uint8_t *buffer,
                                uint32_t size, uint64_t now_ms,
                                mqtt_store_fault *fault);

                                                    See include/base_server.h
```

Parameter	Meaning
`server` *in, out*	The server, set up with a store and holding nothing.
`source` *in*	The store, with from 1 to `MQTT_SERVER_RESTORE_COPIES` (8) copies (Table 4.6).
`map` *out*	Room for the table of store numbers: the storage's `message_count` and `delivery_count` entries together always suffice.
`map_count` *in*	The number of entries in `map`.
`buffer` *out*	Room for the reads, at least `MQTT_SERVER_RESTORE_MINIMUM` (8,192) bytes. Every record but a message has to fit in it, which any does in `MQTT_SERVER_RESTORE_SPACE`.
`size` *in*	The number of bytes in `buffer`.
`now_ms` *in*	The time.
`fault` *out*	What was wrong, when the result is `MQTT_STORE_DAMAGED`.
Returns	`MQTT_OK`; `MQTT_STORE_DAMAGED`; `MQTT_INVALID_ARGUMENT` if the server isn't set up, keeps nothing, or an argument is wrong; or `MQTT_INVALID_STATE` if the server already holds something.

It is all or nothing. If anything is wrong, the server is set up again, empty, and the program sets the store aside: a store from a later version, or one that doesn't fit this configuration or this memory, can then be read by a server that can take it. A

api-reference-en.tex Page 38 November 8, 2026 13:41

store with nothing in it gives `MQTT_OK` and changes nothing. Times come back as what was left of them, less the source's `downtime_ms`. Sessions come back without their connections: one that had a connection when the store was last written counts as having lost it then. Nothing is written until `mqtt_server_snapshot_begin` (§2.10.7), which `mqtt_server_snapshot_due` (§2.10.6) then asks for.

2.10.2 `mqtt_server_store_commit`

Closes the records written since the last call into a group, with a COMMIT record. The answers that wait for them go once the program reports the group stored with `mqtt_server_stored` (§2.10.3). The COMMIT says, by its durability, whether the group needs a sync first. A loop calls this once a round, after the events, so that one sync covers everything that came in the round.

```
mqtt_status mqtt_server_store_commit(mqtt_server *server, uint64_t now_ms,
                                     uint64_t *sequence);
```

See include/base_server.h

Parameter	Meaning
`server` *in, out*	The server.
`now_ms` *in*	The time.
`sequence` *out*	The last group closed, which is the one to report.
Returns	`MQTT_OK`, also when there was nothing to close; `MQTT_INVALID_ARGUMENT` if the server isn't set up or keeps nothing; or `MQTT_INVALID_STATE` if the journal is broken, when `sequence` is the last group closed before it broke.

api-reference-en.tex Page 39 November 8, 2026 13:41

2.10.3 mqtt_server_stored

Reports that every group up to one is stored as its COMMIT asked, so that the answers waiting for them can go. Each connection that can send again is handed out by `mqtt_server_next_pending` (§2.7.11).

```
void mqtt_server_stored(mqtt_server *server, uint64_t sequence);
```

See include/base_server.h

Parameter	Meaning
`server` *in, out*	The server.
`sequence` *in*	The last group stored. One above the last group closed counts as that one.
Returns	Nothing.

2.10.4 mqtt_server_store_failed

Reports that records may have been lost, as when a sync fails. The journal is then broken: nothing more is written to it, and every answer waits, until a snapshot of a new generation is whole.

```
void mqtt_server_store_failed(mqtt_server *server);
```

See include/base_server.h

Parameter	Meaning
`server` *in, out*	The server.
Returns	Nothing.

api-reference-en.tex Page 40 November 8, 2026 13:41

2.10.5 `mqtt_server_store_open`

Finds whether records have been written that no COMMIT has closed yet. A loop that finds some calls `mqtt_server_store_commit` (§2.10.2) before it waits.

```
bool mqtt_server_store_open(const mqtt_server *server);
```

See include/base_server.h

Parameter	Meaning
`server` *in*	The server.
Returns	true if there are some.

2.10.6 `mqtt_server_snapshot_due`

Finds whether a snapshot should begin: none has been whole in this generation, the journal segment has grown past `store_snapshot_bytes`, or the journal is broken.

```
bool mqtt_server_snapshot_due(const mqtt_server *server);
```

See include/base_server.h

Parameter	Meaning
`server` *in*	The server.
Returns	true if one should, and none is under way.

api-reference-en.tex Page 41 November 8, 2026 13:41

2.10.7 `mqtt_server_snapshot_begin`

Begins a snapshot, and with it the journal segment of a new generation. Every record after this call belongs to the new segment, so the program starts that segment with the journal's header before it lets the server run again. Every group before it must have been closed with `mqtt_server_store_commit` (§2.10.2).

```
mqtt_status mqtt_server_snapshot_begin(mqtt_server *server, uint64_t now_ms,
                                       uint8_t *snapshot_header,
                                       uint8_t *journal_header,
                                       bool *new_segment);
```

See include/base_server.h

Parameter	Meaning
`server` *in, out*	The server.
`now_ms` *in*	The time.
`snapshot_header` *out*	Room for the `MQTT_STORE_HEADER_SIZE` (58) bytes of the snapshot's header.
`journal_header` *out*	Room for the `MQTT_STORE_HEADER_SIZE` bytes of the journal segment's header.
`new_segment` *out*	true if a new journal segment starts.
Returns	`MQTT_OK`; `MQTT_INVALID_ARGUMENT` if the server isn't set up or keeps nothing; or `MQTT_INVALID_STATE` if a group is still open or a snapshot is under way.

A snapshot that was aborted is taken again in the same generation, which goes on with the same segment: `new_segment` is then false, and the journal's header isn't written. After a gap in the journal, the new segment is marked as following one, and no answer goes until the snapshot is whole.

api-reference-en.tex Page 42 November 8, 2026 13:41

2.10.8 `mqtt_server_snapshot_next`

Gives the next record of the snapshot under way. The server may run between two calls, and the snapshot stays right, as each record says what something is, and what changes meanwhile is in the journal. The records of one Session come one after another, and if the journal is written to before they are all given, that Session starts again: a loop with a budget for each round stops where `between` is true.

```
mqtt_status mqtt_server_snapshot_next(mqtt_server *server, uint8_t *space,
                                      uint32_t size, mqtt_store_record *record,
                                      bool *between);
```

See include/base_server.h

Parameter	Meaning
`server` *in, out*	The server.
`space` *out*	Where the record's fixed fields and a Topic Filter go, valid until the next call.
`size` *in*	The number of bytes in `space`, at least `MQTT_SERVER_SNAPSHOT_SPACE` (128). With 65,535 more, any record fits.
`record` *out*	The record (Table 4.5). Its runs are valid until the next call.
`between` *out*	true if the next call starts a new message or Session.
Returns	`MQTT_OK` with a record; `MQTT_NOT_FOUND` once the last has been given; `MQTT_BUFFER_TOO_SMALL` if the record doesn't fit in `space`, when the snapshot has to be aborted; or `MQTT_INVALID_STATE` if no snapshot is under way.

api-reference-en.tex Page 43 November 8, 2026 13:41

2.10.9 `mqtt_server_snapshot_done`

Reports that the snapshot is whole in every copy, so that the next one begins a new generation. After a gap in the journal, the answers that waited for it go once their groups are stored.

```
void mqtt_server_snapshot_done(mqtt_server *server);
```

See include/base_server.h

Parameter	Meaning
`server` *in, out*	The server.
Returns	Nothing.

2.10.10 `mqtt_server_snapshot_abort`

Gives up the snapshot under way. The next one is of the same generation, and goes on with the same journal segment.

```
void mqtt_server_snapshot_abort(mqtt_server *server);
```

See include/base_server.h

Parameter	Meaning
`server` *in, out*	The server.
Returns	Nothing.

2.11 Checking a running Server

A server counts what happens as it runs, in the `mqtt_server_statistics` of its `statistics` field, which Table 2.6 lists. They are counts to read, and never to write.

api-reference-en.tex Page 44 November 8, 2026 13:41

Table 2.6 The fields of `mqtt_server_statistics`

Field	What it counts
`connections_accepted`	Connections accepted by a CONNACK.
`messages_received`	Application Messages received from Clients.
`messages_sent`	PUBLISH packets sent to Clients.
`messages_dropped`	Messages dropped because a queue or a pool was full.
`protocol_errors`	Connections closed for an error in a packet.
`messages_refused`	Messages at QoS 0 that couldn't be taken, as the Client that sent one can't be told.
`replies_lost`	Packets saying why a connection closed that there was no room to send.
`receive_throttled`	Times a connection stopped being read for want of room.
`records_written`	Records handed to `store_append`.
`records_refused`	Records `store_append` refused, each of which broke the journal.

2.11.1 `mqtt_server_pool_audit`

Checks that every block of the pool is either free or in exactly one ring of a connection in use, and that the counts agree. It walks every block, so it is for tests and for looking into a fault, not for every round.

```
bool mqtt_server_pool_audit(const mqtt_server *server, uint8_t *marks);
```

See include/base_server.h

Parameter	Meaning
`server` *in*	The server.
`marks` *in, out*	One byte for each block of the pool, all zero.
Returns	true if they agree.

3

The Client

A Client connects to one Server at a time, with MQTT 3.1, 3.1.1 or 5.0. It keeps its Session State from one network connection to the next, as long as the same `mqtt_client` is used: QoS 1 and 2 messages that were not acknowledged go again when `mqtt_client_connect` (§3.6.1) resumes the Session. With a store, it keeps it across a restart too (Section 3.10). It tells the program what the Server sends through one callback.

3.1 At a glance

Table 3.1 The functions of a Client

Function	What it does	§
Setting up		
`mqtt_client_init`	Sets up a client	3.5.1
`mqtt_client_assume_session`	Says it has a Session from an earlier run	3.5.2
Connecting		
`mqtt_client_connect`	Sends a CONNECT	3.6.1
`mqtt_client_auth`	Sends an AUTH	3.6.2
`mqtt_client_disconnect`	Sends a DISCONNECT, and starts to close	3.6.3
`mqtt_client_should_close`	Says when to close the socket	3.6.4
`mqtt_client_connection_lost`	Reports that the connection is gone	3.6.5
Messages and subscriptions		
`mqtt_client_publish`	Sends an Application Message	3.7.1

continues

api-reference-en.tex Page 46 November 8, 2026 13:41

Table 3.1 The functions of a Client (*continued*)

Function	What it does	§
`mqtt_client_subscribe`	Sends a SUBSCRIBE	3.7.2
`mqtt_client_unsubscribe`	Sends an UNSUBSCRIBE	3.7.3
Running with `platform/`		
`mqtt_net_client_open`	Opens the network connection	3.8.1
`mqtt_net_client_step`	Moves the bytes, waiting a little	3.8.2
`mqtt_net_client_close`	Closes the network connection	3.8.3
Running in a loop of the program's own		
`mqtt_client_receive_capacity`	Says how many bytes it may be given	3.9.1
`mqtt_client_receive_space`	Gives the place to read bytes into	3.9.2
`mqtt_client_receive_commit`	Takes the bytes read into it	3.9.3
`mqtt_client_receive`	Takes bytes from a buffer of the program's	3.9.4
`mqtt_client_transmit_view`	Gives the bytes to send	3.9.5
`mqtt_client_transmit_consume`	Says how many of them went	3.9.6
`mqtt_client_tick`	Runs the timers	3.9.7
Keeping Session State		
`mqtt_client_store_init`	Gives a client a store	3.10.1
`mqtt_net_client_store_config`	Fills its configuration for a store of `platform/`	3.10.2
`mqtt_net_client_restore`	Puts back what that store holds	3.10.3
`mqtt_client_store_identifier`	Gives the Client Identifier it holds	3.10.4
`mqtt_client_restore`	Puts back what a store holds, on any medium	3.11.1
`mqtt_client_store_commit`	Closes the records of a round into a group	3.11.2
`mqtt_client_stored`	Says groups are stored, so their answers go	3.11.3
`mqtt_client_store_failed`	Says records may have been lost	3.11.4
`mqtt_client_store_open`	Says whether records wait to be closed	3.11.5
`mqtt_client_snapshot_due`	Says whether a snapshot should begin	3.11.6
`mqtt_client_snapshot_begin`	Begins a snapshot	3.11.7
`mqtt_client_snapshot_next`	Gives its next record	3.11.8

continues

api-reference-en.tex Page 47 November 8, 2026 13:41

Table 3.1 The functions of a Client (*continued*)

Function	What it does	§
`mqtt_client_snapshot_done`	Says its first copy is whole	3.11.9
`mqtt_client_snapshot_abort`	Gives it up	3.11.10

3.2 The life of a Client

1. Give the memory, and the callback for events, in an `mqtt_client_storage` (Section 3.3), and set the client up with `mqtt_client_init` (§3.5.1).
2. To keep its Session State across a restart, give it a store, and put back what the store holds (Section 3.10).
3. Open a network connection: `mqtt_net_client_open` (§3.8.1), or the program's own.
4. Send a CONNECT with `mqtt_client_connect` (§3.6.1), and move bytes until the `MQTT_CLIENT_EVENT_CONNACK` event says whether it was accepted.
5. Publish, subscribe and unsubscribe. Each answer comes as an event.
6. End with `mqtt_client_disconnect` (§3.6.3), move bytes until it is time to close, and close: `mqtt_net_client_close` (§3.8.3), or `mqtt_client_connection_lost` (§3.6.5) in a loop of the program's own.
7. To go on later, open a new connection and send a CONNECT again with the same client.

3.3 The memory

An `mqtt_client_storage` names each array the client will use, and the callback that gets its events. Listing `client_setup.c` in Section 5.2 fills one in.

api-reference-en.tex Page 48 November 8, 2026 13:41

Table 3.2 The fields of `mqtt_client_storage`

Field	What to give
`on_event`, `user_tag`	The callback for events, or `NULL` to ignore them, and the tag handed to it (Section 3.4).
`receive_bytes`, `receive_size`	At least 5 bytes, for a packet that arrives in parts. Its size is the largest packet the Client can take, and is sent to the Server as Maximum Packet Size.
`pool_bytes`, `pool_next`	`pool_block_count` times `pool_block_size` bytes, at least 256 in all, and one `uint32_t` for each block: the bytes waiting to be sent.
`pool_block_size`, `pool_block_count`	Bytes in a block, at most 4,096, and the number of blocks.
`messages`, `message_count`	One `mqtt_client_message` for each message held until it is acknowledged.
`incoming`, `incoming_count`	One `uint16_t` for each QoS 2 message from the Server in progress at once: its Receive Maximum.
`pending`, `pending_count`	One `mqtt_client_pending` for each SUBSCRIBE or UNSUBSCRIBE waiting for its answer.
`aliases`, `alias_count`	One `mqtt_client_alias` for each Topic Alias the Server may set: its Topic Alias Maximum. `NULL` and 0 for none.
`arena_bytes`, `arena_states`, `arena_units`	`arena_units` times `MQTT_ARENA_UNIT` (64) bytes, and one byte for each unit, for the packets held and the Topic Names of Topic Aliases.

3.4 Events

What the Server sends reaches the program as events, through the callback of the storage. It is called during the call that took the bytes in: `mqtt_net_client_step` (§3.8.2), or `mqtt_client_receive_commit` (§3.9.3) in a loop of the program's own.

```
typedef void (*mqtt_client_event_function)(uint32_t user_tag,
                                           const mqtt_client_event *event);
```

See include/base_client.h

The event's `type` says which of its fields is set (Table 3.3). Those pointers, and every byte they lead to, are only valid during the callback: copy what is needed later. The callback may call `mqtt_client_publish` (§3.7.1), `mqtt_client_subscribe`

api-reference-en.tex Page 49 November 8, 2026 13:41

(§3.7.2) and the others that queue packets.

Table 3.3 The events of a Client

Type	Field set	When
`MQTT_CLIENT_EVENT_CONNACK`	`connack`	The CONNACK came. Its `reason_code` is 0 if the connection was accepted; its `session_present` and Properties are there too.
`MQTT_CLIENT_EVENT_AUTH`	`reason`	An AUTH came, in enhanced authentication.
`MQTT_CLIENT_EVENT_MESSAGE`	`publish`	An Application Message came: its `topic_name`, `payload`, `qos`, `retain` and Properties. The acknowledgement is the library's to send.
`MQTT_CLIENT_EVENT_PUBLISHED`	`response`	A QoS 1 or 2 message of this Client's has ended its flow: the PUBACK or PUBCOMP, with its `packet_identifier` and `reason_code`, or a refusal.
`MQTT_CLIENT_EVENT_SUBACK`	`subscribed`	A SUBACK came: one Reason Code for each Topic Filter, in `reason_codes`.
`MQTT_CLIENT_EVENT_UNSUBACK`	`subscribed`	An UNSUBACK came.
`MQTT_CLIENT_EVENT_PINGRESP`	–	A PINGRESP came.
`MQTT_CLIENT_EVENT_DISCONNECT`	`reason`	The Server sent a DISCONNECT, with its Reason Code.

A Reason Code of `MQTT5_RC_FAILURE_MIN` (0x80) or more is a failure, in every version that has Reason Codes in that packet.

api-reference-en.tex Page 50 November 8, 2026 13:41

3.5 Setting up

3.5.1 `mqtt_client_init`

Sets up a client. The description of the memory is copied: the arrays themselves are the client's from now on.

```
mqtt_status mqtt_client_init(mqtt_client *client,
                             const mqtt_client_storage *storage);

                                                    See include/base_client.h
```

Parameter	Meaning
`client` *out*	The client.
`storage` *in*	The memory, and the callback.
Returns	`MQTT_OK`, or `MQTT_INVALID_ARGUMENT` if the storage breaks a rule of Table 3.2.

3.5.2 `mqtt_client_assume_session`

Says that this Client has a Session from an earlier run, with nothing in flight. Without it, a CONNACK whose Session Present is 1 closes the connection, as the protocol asks of a Client with no Session State. A program with no store that starts again and wants to keep the subscriptions it made before calls it before `mqtt_client_connect` (§3.6.1). A client with a store needs none of it: `mqtt_net_client_restore` (§3.10.3) puts its Session back, messages in flight and all.

```
void mqtt_client_assume_session(mqtt_client *client);

                                                    See include/base_client.h
```

Parameter	Meaning
`client` *in, out*	The client.
Returns	Nothing.

api-reference-en.tex Page 51 November 8, 2026 13:41

3.6 Connecting

3.6.1 mqtt_client_connect

Queues a CONNECT on a new network connection. Unless the packet sets them, it adds the Receive Maximum, Maximum Packet Size and Topic Alias Maximum that the storage can honour. The connection is accepted when the MQTT_CLIENT_EVENT_CONNACK event says so.

```
mqtt_status mqtt_client_connect(mqtt_client *client, const mqtt_connect *packet,
                                uint64_t now_ms);
```

See include/base_client.h

Parameter	Meaning
client *in, out*	The client, with no connection in progress.
packet *in*	The CONNECT (Table 3.4). The bytes it refers to are not used after the call.
now_ms *in*	The time.
Returns	MQTT_OK; MQTT_INVALID_STATE if a connection is already in progress; or MQTT_BUFFER_TOO_SMALL if there is no room for it.

Start from mqtt_connect_clear (§4.3.2), which leaves out every optional part, then set what the connection needs.

Table 3.4 The fields of mqtt_connect

Field	Meaning
version	MQTT_VERSION_31, MQTT_VERSION_311 or MQTT_VERSION_5.
clean_start	Clean Start in MQTT 5.0, Clean Session before it.
keep_alive	Keep Alive, in seconds, or 0 for none.
client_identifier	The Client Identifier. Empty, in 3.1.1 and 5.0, asks the Server to assign one.
user_name_flag, user_name	A User Name, if the flag is set.
password_flag, password	A Password, as Binary Data, if the flag is set.

continues

api-reference-en.tex Page 52 November 8, 2026 13:41

Table 3.4 The fields of `mqtt_connect` (*continued*)

Field	Meaning
`will_flag`, `will_topic`, `will_payload`, `will_qos`, `will_retain`	A Will Message, if the flag is set.
`properties`	The CONNECT Properties of MQTT 5.0 (Section 4.2).
`will_properties`	The Will Properties of MQTT 5.0.

3.6.2 `mqtt_client_auth`

Queues an AUTH, for enhanced authentication in MQTT 5.0, when the CONNECT had an Authentication Method: with Reason Code 0x18, Continue Authentication, or 0x19, Re-authenticate, once the connection is accepted. The Authentication Method in its Properties must be the CONNECT's.

```
mqtt_status mqtt_client_auth(mqtt_client *client, const mqtt_reason *packet);
```

See include/base_client.h

Parameter	Meaning
`client` *in, out*	The client.
`packet` *in*	The AUTH: its Reason Code and Properties.
Returns	`MQTT_OK`, or why it could not be queued, as `MQTT_INVALID_STATE`.

api-reference-en.tex Page 53 November 8, 2026 13:41

3.6.3 mqtt_client_disconnect

Queues a DISCONNECT, and starts to close. Keep moving bytes until `mqtt_client_should_close` (§3.6.4) says to close.

```
mqtt_status mqtt_client_disconnect(mqtt_client *client,
                                   const mqtt_reason *packet);

                                                    See include/base_client.h
```

Parameter	Meaning
`client` *in, out*	The client.
`packet` *in*	The DISCONNECT, or `NULL` for a normal one. With MQTT 5.0, Reason Code 0x04 asks the Server to publish the Will Message.
Returns	`MQTT_OK`, or why it could not be queued, in which case it closes anyway.

3.6.4 mqtt_client_should_close

Finds whether the connection should now be closed.

```
bool mqtt_client_should_close(const mqtt_client *client);

                                                    See include/base_client.h
```

Parameter	Meaning
`client` *in*	The client.
Returns	true if it is closing and has nothing left to send.

api-reference-en.tex Page 54 November 8, 2026 13:41

3.6.5 `mqtt_client_connection_lost`

Reports that the network connection has gone, whether the program closed it or not. The Session State stays, for the next `mqtt_client_connect` (§3.6.1). `mqtt_net_client_close` (§3.8.3) calls it.

```
void mqtt_client_connection_lost(mqtt_client *client, uint64_t now_ms);
```

See include/base_client.h

Parameter	Meaning
`client` *in, out*	The client.
`now_ms` *in*	The time.
Returns	Nothing.

3.7 Messages and subscriptions

3.7.1 `mqtt_client_publish`

Sends an Application Message. Its QoS, Topic Name and Properties are checked here, and it is copied, so the bytes can be reused when it returns.

```
mqtt_status mqtt_client_publish(mqtt_client *client, mqtt_publish *packet,
                                uint64_t now_ms);
```

See include/base_client.h

Parameter	Meaning
`client` *in, out*	The client.
`packet` *in, out*	The PUBLISH (Table 3.5). Its Packet Identifier is set here, for a QoS above 0.
`now_ms` *in*	The time.
Returns	`MQTT_OK`, or why it was not taken, in which case nothing has been kept. `MQTT_QUOTA_EXCEEDED` means no room for now: try again once some messages have ended their flow.

api-reference-en.tex Page 55 November 8, 2026 13:41

A QoS 0 message goes only on a connection that has been accepted. A QoS 1 or 2 message is held until its flow ends, and may be given while offline, once `mqtt_client_connect` (§3.6.1) has been called at least once, so that the version is known: it goes with the next connection, unless that Server's limits rule it out, when `MQTT_CLIENT_EVENT_PUBLISHED` reports it with the Reason Code of the limit. A QoS 1 or 2 message must have a Topic Name, even with a Topic Alias, as it may go again on another connection.

Table 3.5 The fields of `mqtt_publish`

Field	Meaning
`topic_name`	The Topic Name: no wildcards.
`payload`	The Payload, of any bytes.
`qos`	0, 1 or 2, no more than the Server's Maximum QoS.
`retain`	true to make it the retained message of its topic.
`dup`	Leave it false: the library sets it on a resend.
`packet_identifier`	Set by the library.
`properties`	The PUBLISH Properties of MQTT 5.0, as Content Type or Message Expiry Interval (Section 4.2).

An `mqtt_publish` whose bytes are all zero has no Properties, so a program can clear one with `memset` and set what it needs, as `client_publish.c` does.

api-reference-en.tex Page 56 November 8, 2026 13:41

3.7.2 mqtt_client_subscribe

Queues a SUBSCRIBE, once the connection has been accepted. The SUBACK comes as the event MQTT_CLIENT_EVENT_SUBACK, with the same Packet Identifier.

```
mqtt_status mqtt_client_subscribe(mqtt_client *client,
                                  const mqtt_client_subscription *entries,
                                  uint32_t count,
                                  const mqtt5_properties *properties,
                                  uint16_t *packet_identifier);
```

See include/base_client.h

Parameter	Meaning
client *in, out*	The client.
entries *in*	The Topic Filters, each with its options (Table 3.6).
count *in*	The number of entries, at least 1.
properties *in*	The SUBSCRIBE Properties of MQTT 5.0, as a Subscription Identifier, or NULL for none.
packet_ identifier *out*	The Packet Identifier the answer will have.
Returns	MQTT_OK; MQTT_INVALID_STATE before the connection is accepted; or why it was not taken, as MQTT_TOPIC_FILTER_INVALID.

An mqtt_client_subscription holds a topic_filter, the Topic Filter, and its options: the Subscription Options byte of MQTT 5.0, or the Requested QoS alone before it:

api-reference-en.tex Page 57 November 8, 2026 13:41

Table 3.6 The bits of `options`, from `v5_subscribe.h`

Bits	Macro	Meaning
0–1	`MQTT5_MAXIMUM_QOS_MASK`	The highest QoS to be sent with.
2	`MQTT5_NO_LOCAL`	Not this Client's own messages.
3	`MQTT5_RETAIN_AS_PUBLISHED`	Keep the RETAIN flag as it was published.
4–5	`MQTT5_RETAIN_HANDLING_MASK`	Whether retained messages are sent: 0 on every subscription, 1 on a new one only, 2 never.

3.7.3 `mqtt_client_unsubscribe`

Queues an UNSUBSCRIBE, once the connection has been accepted. The UNSUBACK comes as the event `MQTT_CLIENT_EVENT_UNSUBACK`.

```
mqtt_status mqtt_client_unsubscribe(mqtt_client *client,
                                    const mqtt_utf8 *filters, uint32_t count,
                                    const mqtt5_properties *properties,
                                    uint16_t *packet_identifier);
```

See include/base_client.h

Parameter	Meaning
`client` *in, out*	The client.
`filters` *in*	The Topic Filters.
`count` *in*	The number of entries, at least 1.
`properties` *in*	The UNSUBSCRIBE Properties of MQTT 5.0, or `NULL` for none.
`packet_identifier` *out*	The Packet Identifier the answer will have.
Returns	`MQTT_OK`, `MQTT_INVALID_STATE` before the connection is accepted, or why it was not taken.

api-reference-en.tex Page 58 November 8, 2026 13:41

3.8 Running with `platform/`

3.8.1 `mqtt_net_client_open`

Opens a client's network connection, over TCP, with TLS if asked. The client stays offline until `mqtt_client_connect` (§3.6.1) is called.

```
mqtt_status mqtt_net_client_open(mqtt_net_client *network, mqtt_client *client,
                                 const char *host, const char *port,
                                 const mqtt_net_client_tls *tls);
```

See include/base_net.h

Parameter	Meaning
`network` *out*	The state of the connection, for the calls below.
`client` *in, out*	The client, which `mqtt_client_init` (§3.5.1) has set up.
`host` *in*	The host to connect to, as a name or an address.
`port` *in*	The port, as text, as `"1883"`.
`tls` *in*	What TLS to use (Table 3.7).
Returns	`MQTT_OK`, `MQTT_CONNECTION_CLOSED`, or `MQTT_TLS_ERROR`.

api-reference-en.tex Page 59 November 8, 2026 13:41

Table 3.7 The fields of `mqtt_net_client_tls`

Field	Meaning
`enabled`	true to use TLS. All the rest is read only then.
`ca_file`	The certificates to trust.
`insecure`	true not to check the Server's certificate, when there is no `ca_file`. Anyone between the two can then read and change everything.
`server_name`	The name to expect in the Server's certificate, or `NULL` for `host`.
`certificate_file`, `key_file`, `key_password`	The client's certificate chain, its key, and the key's password, or `NULL`.

The certificates are loaded once in a program: a later connection keeps what the first loaded.

3.8.2 `mqtt_net_client_step`

Moves what can be moved: runs the timers, reads what came, which raises the events, and writes what waits, waiting up to the time given if there is nothing. For a client with a store of `platform/`, it closes and syncs the records of what came before it writes, so that the answers that waited for them go, and it doesn't wait while records are open.

```
mqtt_status mqtt_net_client_step(mqtt_net_client *network, uint32_t wait_ms);
```

See include/base_net.h

Parameter	Meaning
`network` *in, out*	The client's network connection.
`wait_ms` *in*	The longest to wait, in milliseconds.
Returns	`MQTT_OK`, or `MQTT_CONNECTION_CLOSED` once the connection has gone or should be closed: then call `mqtt_net_client_close` (§3.8.3).

api-reference-en.tex Page 60 November 8, 2026 13:41

3.8.3 mqtt_net_client_close

Closes a client's network connection, and reports it to the client with `mqtt_client_connection_lost` (§3.6.5). A client with a store closes and syncs its last records first.

```
void mqtt_net_client_close(mqtt_net_client *network);
```

See include/base_net.h

Parameter	Meaning
`network` *in, out*	The client's network connection.
Returns	Nothing.

3.9 Running in a loop of the program's own

A program that moves the bytes itself opens the connection, calls `mqtt_client_connect` (§3.6.1), and then, in each round:

1. Calls `mqtt_client_tick` (§3.9.7), which runs the timers and says how long it may wait.
2. Writes the runs of `mqtt_client_transmit_view` (§3.9.5), and calls `mqtt_client_transmit_consume` (§3.9.6) with what the socket took.
3. Reads into the place of `mqtt_client_receive_space` (§3.9.2), and calls `mqtt_client_receive_commit` (§3.9.3) with what was read. The events come during that call.
4. With a store, closes the round's records with `mqtt_client_store_commit` (§3.11.2), keeps them, and reports them with `mqtt_client_stored` (§3.11.3) (Section 3.11).
5. Closes when `mqtt_client_should_close` (§3.6.4) says so, or the socket fails, and calls `mqtt_client_connection_lost` (§3.6.5).

api-reference-en.tex Page 61 November 8, 2026 13:41

3.9.1 mqtt_client_receive_capacity

Finds how many bytes the client may be given now.

```
uint32_t mqtt_client_receive_capacity(const mqtt_client *client);

                                                    See include/base_client.h
```

Parameter	Meaning
`client` *in*	The client.
Returns	The number of bytes.

3.9.2 mqtt_client_receive_space

Finds where the next bytes from the Server can go: the rest of the receive buffer, as far as there is room for the replies.

```
uint32_t mqtt_client_receive_space(const mqtt_client *client,
                                   mqtt_space *space);

                                                    See include/base_client.h
```

Parameter	Meaning
`client` *in*	The client.
`space` *out*	The place.
Returns	The number of bytes it holds, or zero if there is no room now.

api-reference-en.tex Page 62 November 8, 2026 13:41

3.9.3 mqtt_client_receive_commit

Takes the bytes written to the place `mqtt_client_receive_space` (§3.9.2) gave. Each whole packet is handled where it is, which raises its event, and the start of one that has only partly arrived is moved to the front of the buffer.

```
mqtt_status mqtt_client_receive_commit(mqtt_client *client, uint32_t count,
                                       uint64_t now_ms);
```

See include/base_client.h

Parameter	Meaning
`client` *in, out*	The client.
`count` *in*	The number of bytes written. No more than the place holds are counted.
`now_ms` *in*	The time.
Returns	`MQTT_OK`, or the reason the connection is now closing.

3.9.4 mqtt_client_receive

Takes bytes that arrived from a buffer of the program's.

```
mqtt_status mqtt_client_receive(mqtt_client *client, const uint8_t *data,
                                uint32_t length, uint64_t now_ms);
```

See include/base_client.h

Parameter	Meaning
`client` *in, out*	The client.
`data` *in*	The bytes.
`length` *in*	The number of bytes in `data`.
`now_ms` *in*	The time.
Returns	`MQTT_OK`, or the reason the connection is now closing.

api-reference-en.tex Page 63 November 8, 2026 13:41

3.9.5 mqtt_client_transmit_view

Finds the next bytes to send, as runs that one `writev` can send together.

```
uint32_t mqtt_client_transmit_view(mqtt_client *client, mqtt_run *runs,
                                   uint32_t max_runs);
```

See include/base_client.h

Parameter	Meaning
`client` *in, out*	The client.
`runs` *out*	The runs, in the order they go, valid until the next call about this client.
`max_runs` *in*	The number of entries `runs` can hold. One for each block of the ring, and one more, is enough for everything that waits; one gives the first run alone.
Returns	The number of runs, or zero if there is nothing to send now.

3.9.6 mqtt_client_transmit_consume

Reports how many bytes of the view have been sent, from its first run on.

```
void mqtt_client_transmit_consume(mqtt_client *client, uint32_t count);
```

See include/base_client.h

Parameter	Meaning
`client` *in, out*	The client.
`count` *in*	The number of bytes sent, which may end in any run. No more than are waiting are taken.
Returns	Nothing.

api-reference-en.tex Page 64 November 8, 2026 13:41

3.9.7 `mqtt_client_tick`

Runs the timers: Keep Alive, which sends a PINGREQ when it is due, and the wait for a CONNACK.

```
uint32_t mqtt_client_tick(mqtt_client *client, uint64_t now_ms);
```

See include/base_client.h

Parameter	Meaning
`client` *in, out*	The client.
`now_ms` *in*	The time.
Returns	The number of milliseconds until it should run again.

3.10 Keeping Session State across a restart

A client can keep its Session State in a store as a server does (Section 4.5): whether it has a Session, with which version and Client Identifier, the messages it holds for the Server, the Packet Identifiers of the QoS 2 messages it has received, and with MQTT 3.1 the SUBSCRIBE and UNSUBSCRIBE packets waiting for their answers. Started again on the same store, it is offline with the Session it had: what it held goes out on the next connection that resumes it, with DUP set on a message that was sent.

Once it keeps a store, an answer that tells the Server something was kept waits until its group is stored: PUBACK, PUBREC, PUBREL, PUBCOMP, a QoS 2 PUBLISH, CONNECT, and with MQTT 3.1 SUBSCRIBE and UNSUBSCRIBE. A message from the Server reaches the event callback before the answer to it is sent, so a program that keeps what it is given there loses none of it.

api-reference-en.tex Page 65 November 8, 2026 13:41

Table 3.8 The fields of `mqtt_client_store_config`

Field	Meaning
`append`	Hands each record of the journal to the program, to keep.
`user_tag`	The value given to `append`. The storage's `user_tag` goes to the events instead.
`sessions`	What the records of the Session have to outlast (Table 4.4): its version, whether it exists, and the Client Identifier. `MQTT_STORE_MEMORY` writes nothing at all.
`messages`	What the records of messages have to outlast: those held for the Server, the QoS 2 Packet Identifiers received, and with MQTT 3.1 the SUBSCRIBE and UNSUBSCRIBE packets waiting. `MQTT_STORE_MEMORY` keeps the Session without its messages.
`snapshot_bytes`	Bytes of journal after which a snapshot is due.

Nothing fills one with defaults: `mqtt_client_init` (§3.5.1) leaves a client with no store, and `mqtt_net_client_store_config` (§3.10.2) fills one for a store of `platform/`. With `platform/`, a program sets the client up, opens a store with `mqtt_platform_store_open` (§4.5.2), fills the configuration, gives it with `mqtt_client_store_init` (§3.10.1), and puts back what the store holds with `mqtt_net_client_restore` (§3.10.3), all before the first connection. Each `mqtt_net_client_step` (§3.8.2) then does the rest. A program runs one client with a store of `platform/` at a time.

api-reference-en.tex Page 66 November 8, 2026 13:41

3.10.1 mqtt_client_store_init

Gives a client a store, after `mqtt_client_init` (§3.5.1) and before anything is connected or published. Then put back what the store holds, with `mqtt_net_client_restore` (§3.10.3) or `mqtt_client_restore` (§3.11.1). Nothing is written until the first snapshot, which has to be whole before the first connection.

```
mqtt_status mqtt_client_store_init(mqtt_client *client,
                                   const mqtt_client_store_config *config);

                                                        See include/base_client.h
```

Parameter	Meaning
`client` *in, out*	The client.
`config` *in*	The configuration (Table 3.8), which is copied.
Returns	`MQTT_OK`; `MQTT_INVALID_ARGUMENT` if the client isn't set up or the configuration is wrong; or `MQTT_INVALID_STATE` if the client has already been used, or keeps a store already.

3.10.2 mqtt_net_client_store_config

Fills a client's store configuration for a store of `platform/`, for `mqtt_client_store_init` (§3.10.1). Records go to `mqtt_platform_store_append`, with the store as its tag. The Session and the messages are kept to outlast a loss of power, which the program may lower to `MQTT_STORE_WRITTEN` after this call, and a snapshot is due after 1 MiB of journal, or half a place of the store if that is less.

```
void mqtt_net_client_store_config(mqtt_client_store_config *config,
                                  int32_t store);

                                                        See include/base_net.h
```

Parameter	Meaning
`config` *out*	The configuration.
`store` *in*	The store, from `mqtt_platform_store_open` (§4.5.2).
Returns	Nothing.

api-reference-en.tex Page 67 November 8, 2026 13:41

3.10.3 mqtt_net_client_restore

Puts back what a store holds into a client just given it with `mqtt_client_store_init` (§3.10.1), and takes the first snapshot, before the first connection. A store that is damaged, from a later version, or more than this client can take, is set aside untouched, a line on the console says why, and the client starts empty. From then on, each `mqtt_net_client_step` (§3.8.2) closes and syncs the client's records, and lets the answers that waited for them go.

```
mqtt_status mqtt_net_client_restore(mqtt_client *client, int32_t store,
                                    mqtt_store_fault *fault);
```

See include/base_net.h

Parameter	Meaning
`client` *in, out*	The client.
`store` *in*	The store.
`fault` *out*	What was wrong with a store set aside (Table 4.7), or `MQTT_STORE_FAULT_NONE`.
Returns	`MQTT_OK` if the client can connect, restored or empty; `MQTT_INVALID_ARGUMENT` or `MQTT_INVALID_STATE` as `mqtt_client_restore` (§3.11.1) gives them; `MQTT_INVALID_STATE` if a damaged store could not be set aside; or why the first snapshot couldn't be written.

As for a server, `MQTT_OK` comes back after a store was set aside too, and only `fault` tells the two apart.

api-reference-en.tex Page 68 November 8, 2026 13:41

3.10.4 `mqtt_client_store_identifier`

Gives the Client Identifier of the Session the store holds: the one the last CONNECT gave, or the one the Server assigned, so that a program that let the Server assign one can give it in the CONNECT that resumes the Session.

```
mqtt_status mqtt_client_store_identifier(const mqtt_client *client,
                                         mqtt_utf8 *identifier);

                                                    See include/base_client.h
```

Parameter	Meaning
`client` *in*	The client.
`identifier` *out*	The identifier, valid until the next connection.
Returns	`MQTT_OK`, or `MQTT_NOT_FOUND` if none is known.

3.11 A store in a loop of the program's own

A program that keeps the store itself fills an `mqtt_client_store_config` of its own, and gives it with `mqtt_client_store_init` (§3.10.1). A client's snapshot holds only what its memory holds, so it is always taken whole, between two rounds. Then:

1. Put back what the store holds with `mqtt_client_restore` (§3.11.1). If it gives `MQTT_STORE_DAMAGED`, set the store aside, and start it again empty.
2. Before the first connection, and whenever `mqtt_client_snapshot_due` (§3.11.6) says so, take a whole snapshot: call `mqtt_client_snapshot_begin` (§3.11.7), start the first copy with the snapshot's header, add each record of `mqtt_client_snapshot_next` (§3.11.8) until `MQTT_NOT_FOUND`, and sync it. If `new_segment` is true, then empty the journal place of the new generation, and write the journal's header there. Call `mqtt_client_snapshot_done` (§3.11.9), and write the other copies.
3. In each round, once the bytes that came are taken in, close its records with `mqtt_client_store_commit` (§3.11.2), sync if the COMMIT asks for that, and report them with `mqtt_client_stored` (§3.11.3), or call `mqtt_client_store_failed` (§3.11.4) if the sync failed.
4. Don't wait while `mqtt_client_store_open` (§3.11.5) says records are open:

api-reference-en.tex Page 69 November 8, 2026 13:41

close them first.

3.11.1 mqtt_client_restore

Puts back what a store holds, into a client just given its store. Every place is read and checked first, then the newest snapshot copy that its journal segments follow on from is applied, and those segments up to their last COMMIT. Each record is checked as what the Server sends is, and has to fit this client's memory.

```
mqtt_status mqtt_client_restore(mqtt_client *client,
                                const mqtt_store_source *source,
                                uint8_t *buffer, uint32_t size,
                                mqtt_store_fault *fault);

                                                    See include/base_client.h
```

Parameter	Meaning
`client` *in, out*	The client, with a store and holding nothing.
`source` *in*	The store, with from 1 to `MQTT_CLIENT_RESTORE_COPIES` (8) copies (Table 4.6).
`buffer` *out*	Room for the reads, at least `MQTT_CLIENT_RESTORE_MINIMUM` (256) bytes. Packets and the Client Identifier are read straight into the arena.
`size` *in*	The number of bytes in `buffer`.
`fault` *out*	What was wrong, when the result is `MQTT_STORE_DAMAGED`.
Returns	`MQTT_OK`; `MQTT_STORE_DAMAGED`; `MQTT_INVALID_ARGUMENT` if the client isn't set up, keeps nothing, or an argument is wrong; or `MQTT_INVALID_STATE` if the client already holds something.

It is all or nothing. If anything is wrong, the client is set up again, empty, with its store, and the program sets the store aside. A store with nothing in it gives `MQTT_OK` and changes nothing.

api-reference-en.tex Page 70 November 8, 2026 13:41

3.11.2 mqtt_client_store_commit

Closes the records written since the last call into a group, with a COMMIT record. The answers that wait for them go once the program reports the group stored with `mqtt_client_stored` (§3.11.3). A loop calls this once a round, after the bytes that came, so that one sync covers everything they caused.

```
mqtt_status mqtt_client_store_commit(mqtt_client *client, uint64_t now_ms,
                                     uint64_t *sequence);

                                                        See include/base_client.h
```

Parameter	Meaning
`client` *in, out*	The client.
`now_ms` *in*	The time.
`sequence` *out*	The last group closed, which is the one to report.
Returns	`MQTT_OK`, also when there was nothing to close; `MQTT_INVALID_ARGUMENT` if the client isn't set up or keeps nothing; or `MQTT_INVALID_STATE` if the journal is broken.

3.11.3 mqtt_client_stored

Reports that every group up to one is stored as its COMMIT asked, so that the answers waiting for them can go.

```
void mqtt_client_stored(mqtt_client *client, uint64_t sequence);

                                                        See include/base_client.h
```

Parameter	Meaning
`client` *in, out*	The client.
`sequence` *in*	The last group stored.
Returns	Nothing.

api-reference-en.tex Page 71 November 8, 2026 13:41

3.11.4 `mqtt_client_store_failed`

Reports that records may have been lost, as when a sync fails. The journal is then broken: nothing more is written, and every answer waits, until a snapshot of a new generation is whole.

```
void mqtt_client_store_failed(mqtt_client *client);
```

See include/base_client.h

Parameter	Meaning
`client` *in, out*	The client.
Returns	Nothing.

3.11.5 `mqtt_client_store_open`

Finds whether records have been written that no COMMIT has closed yet. A loop that finds some calls `mqtt_client_store_commit` (§3.11.2) before it waits.

```
bool mqtt_client_store_open(const mqtt_client *client);
```

See include/base_client.h

Parameter	Meaning
`client` *in*	The client.
Returns	true if there are some.

api-reference-en.tex Page 72 November 8, 2026 13:41

3.11.6 `mqtt_client_snapshot_due`

Finds whether a snapshot should begin: none has been whole in this generation, the journal segment has grown past `snapshot_bytes`, or the journal is broken.

```
bool mqtt_client_snapshot_due(const mqtt_client *client);
```

See include/base_client.h

Parameter	Meaning
`client` *in*	The client.
Returns	true if one should, and none is under way.

api-reference-en.tex Page 73 November 8, 2026 13:41

3.11.7 mqtt_client_snapshot_begin

Begins a snapshot, which is taken whole: the client must not be called otherwise until `mqtt_client_snapshot_done` (§3.11.9) or `mqtt_client_snapshot_abort` (§3.11.10). The snapshot of a new generation starts a new journal segment, which the program starts with the journal's header once the first copy of the snapshot is whole, and before `mqtt_client_snapshot_done` (§3.11.9). Every group before it must have been closed with `mqtt_client_store_commit` (§3.11.2).

```
mqtt_status mqtt_client_snapshot_begin(mqtt_client *client, uint64_t now_ms,
                                       uint8_t *snapshot_header,
                                       uint8_t *journal_header,
                                       bool *new_segment);

                                                    See include/base_client.h
```

Parameter	Meaning
`client` *in, out*	The client.
`now_ms` *in*	The time.
`snapshot_header` *out*	Room for the `MQTT_STORE_HEADER_SIZE` (58) bytes of the snapshot's header.
`journal_header` *out*	Room for the `MQTT_STORE_HEADER_SIZE` bytes of the journal segment's header.
`new_segment` *out*	true if a new journal segment starts.
Returns	`MQTT_OK`; `MQTT_INVALID_ARGUMENT` if the client isn't set up or keeps nothing; or `MQTT_INVALID_STATE` if a group is still open or a snapshot is under way.

api-reference-en.tex Page 74 November 8, 2026 13:41

3.11.8 `mqtt_client_snapshot_next`

Gives the next record of the snapshot under way.

```
mqtt_status mqtt_client_snapshot_next(mqtt_client *client, uint8_t *space,
                                      uint32_t size,
                                      mqtt_store_record *record);

                                                    See include/base_client.h
```

Parameter	Meaning
`client` *in, out*	The client.
`space` *out*	Where the record's frame and fixed fields go, valid until the next call.
`size` *in*	The number of bytes in `space`, at least `MQTT_CLIENT_SNAPSHOT_SPACE` (32).
`record` *out*	The record (Table 4.5). Its runs are valid until the next call.
Returns	`MQTT_OK` with a record; `MQTT_NOT_FOUND` once the last has been given; `MQTT_BUFFER_TOO_SMALL` if `space` is too small; or `MQTT_INVALID_STATE` if no snapshot is under way, or the client has changed since it began.

3.11.9 `mqtt_client_snapshot_done`

Reports that the snapshot's first copy is whole, and its journal segment started if it starts one, so that the next snapshot begins a new generation. After a gap in the journal, the answers that waited for it go once their group is stored.

```
void mqtt_client_snapshot_done(mqtt_client *client);

                                                    See include/base_client.h
```

Parameter	Meaning
`client` *in, out*	The client.
Returns	Nothing.

api-reference-en.tex Page 75 November 8, 2026 13:41

3.11.10 `mqtt_client_snapshot_abort`

Gives up the snapshot under way.

```
void mqtt_client_snapshot_abort(mqtt_client *client);
```

See include/base_client.h

Parameter	Meaning
`client` *in, out*	The client.
Returns	Nothing.

4

Shared by Both

This chapter has the types and functions the Server and the Client both use: the views of bytes, the Properties of MQTT 5.0, Reason Codes, TLS, the store, what `platform/` offers a program of the system it runs on, and the version of the library.

4.1 Views of bytes

Table 4.1 The views of bytes

Type	Fields	For
`mqtt_utf8`	`length` (16 bits), `data`	A UTF-8 Encoded String: topics, Client Identifiers, string Properties.
`mqtt_binary`	`length` (16 bits), `data`	Binary Data: Password, Will Payload, Correlation Data.
`mqtt_payload`	`length` (32 bits), `data`	A PUBLISH Payload.
`mqtt_run`	`data`, `length`	Bytes to write, from `transmit_view`.
`mqtt_space`	`data`, `length`	Room to read into, from `receive_space`.
`mqtt5_utf8_pair`	`name`, `value`	A User Property.

None of them owns its bytes, and none ends in a NUL: print one with `printf("\%.*s", (int)s.length, (const char *)s.data)`, as `client_subscribe.c` does.

api-reference-en.tex Page 78 November 8, 2026 13:41

4.1.1 `mqtt_utf8_from_text`

Makes a view of a NUL-terminated string, without copying it. The string must outlive the view.

```
mqtt_utf8 mqtt_utf8_from_text(const char *text);

                                                    See include/base_wire.h
```

Parameter	Meaning
`text` *in*	The text, or `NULL` for an empty view.
Returns	The view.

4.2 Properties

The Properties of a packet of MQTT 5.0 are an `mqtt5_properties`: one field for each Property, and a mask that says which are present. Only a Property marked present is sent, or has a value when it is read. Before 5.0, a packet has no Properties, and they are ignored.

- **To send one.** Clear the set with `mqtt5_properties_clear` (§4.2.1) (or start from `mqtt_connect_clear` (§4.3.2), `mqtt_reason_clear` (§4.3.1) or a zeroed packet), set the field, and mark it with `mqtt5_properties_mark` (§4.2.2). User Properties are an array of pairs in `user_pairs` and `user_pair_count`, with no mark.
- **To read one.** Ask `mqtt5_properties_has` (§4.2.3), then read the field.
- **To read every one.** A Property that can come more than once, as User Property, is read from `raw`, the whole block as it came, with a reader: `mqtt_reader_init` on `raw`, then `mqtt5_property_next` (§4.2.4) until it returns false.

```
mqtt5_properties_clear(&properties);
properties.message_expiry_interval = 60U;   /* seconds */
mqtt5_properties_mark(&properties, MQTT5_PROP_MESSAGE_EXPIRY_INTERVAL);

if (mqtt5_properties_has(&event->publish->properties, MQTT5_PROP_CONTENT_TYPE)) {
    mqtt_utf8 type = event->publish->properties.content_type;
}
```

api-reference-en.tex Page 79 November 8, 2026 13:41

Table 4.2 The Properties, with the field of each and its Identifier in `v5_prop.h`

Field	Identifier
`payload_format_indicator`	`MQTT5_PROP_PAYLOAD_FORMAT_INDICATOR`
`message_expiry_interval`	`MQTT5_PROP_MESSAGE_EXPIRY_INTERVAL`
`content_type`	`MQTT5_PROP_CONTENT_TYPE`
`response_topic`	`MQTT5_PROP_RESPONSE_TOPIC`
`correlation_data`	`MQTT5_PROP_CORRELATION_DATA`
`subscription_identifier`	`MQTT5_PROP_SUBSCRIPTION_IDENTIFIER`: the first, if there are several
`session_expiry_interval`	`MQTT5_PROP_SESSION_EXPIRY_INTERVAL`
`assigned_client_identifier`	`MQTT5_PROP_ASSIGNED_CLIENT_IDENTIFIER`
`server_keep_alive`	`MQTT5_PROP_SERVER_KEEP_ALIVE`
`authentication_method`	`MQTT5_PROP_AUTHENTICATION_METHOD`
`authentication_data`	`MQTT5_PROP_AUTHENTICATION_DATA`
`request_problem_information`	`MQTT5_PROP_REQUEST_PROBLEM_INFORMATION`
`will_delay_interval`	`MQTT5_PROP_WILL_DELAY_INTERVAL`
`request_response_information`	`MQTT5_PROP_REQUEST_RESPONSE_INFORMATION`
`response_information`	`MQTT5_PROP_RESPONSE_INFORMATION`
`server_reference`	`MQTT5_PROP_SERVER_REFERENCE`
`reason_string`	`MQTT5_PROP_REASON_STRING`
`receive_maximum`	`MQTT5_PROP_RECEIVE_MAXIMUM`
`topic_alias_maximum`	`MQTT5_PROP_TOPIC_ALIAS_MAXIMUM`
`topic_alias`	`MQTT5_PROP_TOPIC_ALIAS`
`maximum_qos`	`MQTT5_PROP_MAXIMUM_QOS`
`retain_available`	`MQTT5_PROP_RETAIN_AVAILABLE`
`user_pairs`, `user_pair_count`	`MQTT5_PROP_USER_PROPERTY`: set the pairs, no mark
`maximum_packet_size`	`MQTT5_PROP_MAXIMUM_PACKET_SIZE`
`wildcard_subscription_available`	`MQTT5_PROP_WILDCARD_SUBSCRIPTION_AVAILABLE`

continues

api-reference-en.tex Page 80 November 8, 2026 13:41

Table 4.2 The Properties (*continued*)

Field	Identifier
`subscription_identifier_available`	`MQTT5_PROP_SUBSCRIPTION_IDENTIFIER_AVAILABLE`
`shared_subscription_available`	`MQTT5_PROP_SHARED_SUBSCRIPTION_AVAILABLE`

4.2.1 `mqtt5_properties_clear`

Clears a set: marks no Property present, and drops `raw` and the User Properties. The other fields keep what they held, as none is read until it is marked.

```
void mqtt5_properties_clear(mqtt5_properties *properties);
```

See include/v5_properties.h

Parameter	Meaning
`properties` *out*	The Properties.
Returns	Nothing.

4.2.2 `mqtt5_properties_mark`

Marks a Property as present, once its field is set.

```
void mqtt5_properties_mark(mqtt5_properties *properties, uint32_t identifier);
```

See include/v5_properties.h

Parameter	Meaning
`properties` *in, out*	The Properties.
`identifier` *in*	Its Identifier, from `v5_prop.h`.
Returns	Nothing.

api-reference-en.tex Page 81 November 8, 2026 13:41

4.2.3 mqtt5_properties_has

Finds whether a Property is present.

```
bool mqtt5_properties_has(const mqtt5_properties *properties,
                          uint32_t identifier);

                                                    See include/v5_properties.h
```

Parameter	Meaning
`properties` *in*	The Properties.
`identifier` *in*	Its Identifier, from `v5_prop.h`.
Returns	true if it is present.

4.2.4 mqtt5_property_next

Reads the next Property of a block that has been received, which has been checked already: each User Property and each Subscription Identifier comes on its own.

```
bool mqtt5_property_next(mqtt_reader *cursor, mqtt5_property *property);

                                                    See include/v5_properties.h
```

Parameter	Meaning
`cursor` *in, out*	A reader over `raw`, started with `mqtt_reader_init(\&cursor, raw.data, raw.length)`, from `base_wire.h`.
`property` *out*	The Property: its `identifier`, and its value in `integer`, `text`, `binary`, or `text` and `value` for a User Property.
Returns	true if there was one, false at the end of the block.

4.3 Reason Codes

DISCONNECT and AUTH packets are an `mqtt_reason`: a `reason_code` and `properties`. The Reason Codes of MQTT 5.0 are the `MQTT5_RC_` macros of `v5_rc.h`, and the return codes of 3.1.1 and 3.1 those of `v311_rc.h` and `v31_rc.h`.

api-reference-en.tex Page 82 November 8, 2026 13:41

4.3.1 `mqtt_reason_clear`

Clears a DISCONNECT or an AUTH: Reason Code 0, no Properties.

```
void mqtt_reason_clear(mqtt_reason *packet);
```

See include/base_packet.h

Parameter	Meaning
`packet` *out*	The packet.
Returns	Nothing.

4.3.2 `mqtt_connect_clear`

Clears a CONNECT: version 3.1.1, every flag false, Keep Alive 0, every string empty, no Properties. It is cheaper than zeroing it, and the memory may hold anything before.

```
void mqtt_connect_clear(mqtt_connect *packet);
```

See include/base_packet.h

Parameter	Meaning
`packet` *out*	The packet.
Returns	Nothing.

api-reference-en.tex Page 83 November 8, 2026 13:41

4.3.3 mqtt5_reason_code

Finds the Reason Code that reports an `mqtt_status`, as a Server's callback may want to.

```
uint8_t mqtt5_reason_code(mqtt_status status);
```

See include/base_packet.h

Parameter	Meaning
`status` *in*	A status other than `MQTT_OK`.
Returns	The Reason Code, or `MQTT5_RC_UNSPECIFIED_ERROR` if none is closer.

4.4 TLS

TLS is in `platform/`, through Mbed TLS, in a library built with `WITH_TLS=yes`. The loops of `platform/` use it themselves: a Server through the TLS fields of its options, a Client through `mqtt_net_client_tls`. A loop of the program's own can use the same functions, which Table 4.3 lists.

4.4.1 mqtt_tls_available

Finds whether the library was built with TLS. Built without it, every other function of `base_tls.h` reports `MQTT_TLS_ERROR`.

```
bool mqtt_tls_available(void);
```

See include/base_tls.h

Parameter	Meaning
No parameters.	
Returns	true if it was.

api-reference-en.tex Page 84 November 8, 2026 13:41

Table 4.3 The functions of `base_tls.h`

Function	What it does
`mqtt_tls_server_setup`	Loads the Server's certificate chain and key, in PEM form, and the certificates to check clients against, if any.
`mqtt_tls_client_setup`	Loads the certificates a client trusts, and its own, once in a program.
`mqtt_tls_session_open`	Starts a session on a connected socket, for either side. Gives `MQTT_TLS_NONE` with no room for another.
`mqtt_tls_read`, `mqtt_tls_write`	Read and write through a session, as `mqtt_platform_read` and `mqtt_platform_write` do on a socket.
`mqtt_tls_wants_read`, `mqtt_tls_wants_write`	Say whether a session needs the socket readable or writable to go on.
`mqtt_tls_has_pending`	Says whether a session holds bytes already read from the socket, which a wait for the socket would not show.
`mqtt_tls_session_close`	Ends a session and frees it.

4.5 The store

A store keeps what a Server or a Client must not lose across a restart, as records whose meaning is the library's: the program, or `platform/`, only keeps their bytes. `base_store.h` defines them. A store has two parts. The snapshot, the whole state as records, is kept in two copies or more, written one after the other. The journal holds each change made after it, in two places used in turn, one for each generation of the snapshot. Each copy and each journal segment starts with a header of `MQTT_STORE_HEADER_SIZE` (58) bytes, every record carries a CRC-16, and a journal is a series of groups, each closed by a COMMIT record. A restore takes a group whole or not at all, so a write that a crash cut short loses only the group it was in, whose answers never went. A CRC finds damage, not forgery, so a store belongs where only the program can write it.

api-reference-en.tex Page 85 November 8, 2026 13:41

Table 4.4 The values of `mqtt_store_durability`

Value	What a record outlasts before what depends on it goes
`MQTT_STORE_MEMORY`	Nothing: it is not written at all.
`MQTT_STORE_WRITTEN`	A crash of the process: it is written to the medium.
`MQTT_STORE_SYNCED`	A loss of power: it is written, and synced.

A Server's `store_append`, and a Client's `append`, have this type. The bytes of the record (Table 4.5) are only valid during the call, so they are written or copied before it returns. It returns `MQTT_OK`, or anything else if the record could not be added, which breaks the journal until the next snapshot is whole.

```
typedef mqtt_status (*mqtt_store_append)(uint32_t user_tag,
                                         const mqtt_store_record *record);
```

See include/base_store.h

Table 4.5 The fields of `mqtt_store_record`

Field	Meaning
`runs`, `run_count`	The record's bytes, with its frame and CRC, in order, in at most `MQTT_STORE_RECORD_RUNS` (6) runs.
`length`	The number of bytes in all the runs.
`sequence`	The group the record belongs to.
`durability`	What it has to outlast. For a COMMIT, the most any record of its group has to: the group needs a sync if that is `MQTT_STORE_SYNCED`.
`kind`	The kind of record. A COMMIT is `MQTT_STORE_RECORD_COMMIT`.

A restore reads the store through an `mqtt_store_source` (Table 4.6), whose function has this type. It reads from a place of one part, `MQTT_STORE_SNAPSHOT` or `MQTT_STORE_JOURNAL`: a copy of the snapshot, or a place of the journal. `count` is less than `capacity` only at the end of the place, and 0 past it or for a place that doesn't exist. It returns `MQTT_OK`, or anything else if the medium could not be read.

api-reference-en.tex Page 86 November 8, 2026 13:41


```
typedef mqtt_status (*mqtt_store_read)(uint32_t user_tag, mqtt_store_part part,
                                       uint32_t place, uint64_t offset,
                                       uint8_t *data, uint32_t capacity,
                                       uint32_t *count);

                                                    See include/base_store.h
```

Table 4.6 The fields of `mqtt_store_source`

Field	Meaning
`read`, `user_tag`	The function that reads the store, and the value given to it.
`copies`	The number of copies of the snapshot.
`erased`	The value of a byte never written: 0x00 in a file, 0xFF in flash. A record that fails its check counts as cut short by a crash only if every byte after it has this value; otherwise the store is damaged.
`downtime_ms`	Milliseconds from the last write to the restore, or 0 if the medium can't tell. What is left of each interval comes back less this.

A restore that finds the store damaged, or not one it can take, says where and why in an `mqtt_store_fault` (Tables 4.7 and 4.8).

Table 4.7 The fields of `mqtt_store_fault`

Field	Meaning
`kind`	What was wrong (Table 4.8).
`part`, `place`	Where: the part of the store, and the copy or the journal place.
`generation`	The generation of the copy or the segment, or 0 if its header was not read.
`offset`, `record_kind`	The offset of the record in its place, and its kind, or 0 and 0 for the header.

api-reference-en.tex Page 87 November 8, 2026 13:41

Table 4.8 The values of `mqtt_store_fault_kind`

Value	Meaning
`MQTT_STORE_FAULT_NONE`	Nothing.
`MQTT_STORE_FAULT_NO_COPY`	No copy of the snapshot passes its checks.
`MQTT_STORE_FAULT_HEADER`	The header of a copy or a journal segment is damaged, or not one this library writes.
`MQTT_STORE_FAULT_VERSION`	A copy or a journal segment has a newer major version of the format.
`MQTT_STORE_FAULT_RECORD`	A record fails its check, and more bytes follow it.
`MQTT_STORE_FAULT_CONTENT`	A record passes its check, but holds what may not be there.
`MQTT_STORE_FAULT_CHAIN`	The journal segments don't follow on from the snapshot and from each other.
`MQTT_STORE_FAULT_ROOM`	The memory given is too small for what is stored.
`MQTT_STORE_FAULT_MEDIUM`	The medium could not be read.
`MQTT_STORE_FAULT_CONFIG`	A record asks for what the configuration no longer offers, or goes past one of its limits.

4.5.1 `mqtt_store_fault_name`

Names a kind of fault, for a report.

```
const char *mqtt_store_fault_name(mqtt_store_fault_kind kind);
```

See include/base_store.h

Parameter	Meaning
`kind` *in*	The kind of fault.
Returns	Words that describe it, such as `no fault` for `MQTT_STORE_FAULT_NONE`.

`platform/` keeps a store in a folder of files on every system that has files, and in a region of flash on FreeRTOS and uC/OS-III, which the board names and gives the calls for, as *Building libterae* describes. At most `MQTT_PLATFORM_STORES` (2) are open at once: a Server's and a Client's.

api-reference-en.tex Page 88 November 8, 2026 13:41

4.5.2 mqtt_platform_store_open

Opens a store, and makes it if there is none. In a folder, which is made if it isn't there, the store is the files `snapshot.0` and on, one for each copy, `journal.0` and `journal.1`; on a POSIX system only their owner can read or write them. In flash, the region is cut into as many places of whole erase blocks as there are copies, and two more for the journal.

```
mqtt_status mqtt_platform_store_open(const char *where, uint32_t copies,
                                     int32_t *store);

                                                    See include/base_platform.h
```

Parameter	Meaning
`where` *in*	The folder of its files, or the name of the board's region.
`copies` *in*	The number of copies of the snapshot, from `MQTT_PLATFORM_STORE_COPIES_MIN` (2) to `MQTT_PLATFORM_STORE_COPIES_MAX` (8).
`store` *out*	The store, or `MQTT_STORE_NONE` if it was not opened.
Returns	`MQTT_OK`; `MQTT_INVALID_ARGUMENT` if `where` is empty or too long, or `copies` is out of range; `MQTT_QUOTA_EXCEEDED` if `MQTT_PLATFORM_STORES` are open; or `MQTT_INVALID_STATE` if the medium can't be opened, or the system has none.

4.5.3 mqtt_platform_store_close

Closes a store. One that isn't open, `MQTT_STORE_NONE` among them, is ignored, so a program can close its store whatever happened before.

```
void mqtt_platform_store_close(int32_t store);

                                                    See include/base_platform.h
```

Parameter	Meaning
`store` *in*	The store.
Returns	Nothing.

The loops of `platform/` keep their stores with the other calls of Table 4.9. A

api-reference-en.tex Page 89 November 8, 2026 13:41

program with a loop of its own can use them too, to keep its store where `platform/` would.

Table 4.9 The other store calls of `base_platform.h`

Function	What it does
`mqtt_platform_store_append`	A `store_append`, whose tag is the store: adds a record to the journal place in use, and syncs nothing yet.
`mqtt_platform_store_commit`	Syncs the journal, if a COMMIT added since the last call asks for it.
`mqtt_platform_store_read`	The `read` of an `mqtt_store_source`, whose tag is the store.
`mqtt_platform_store_segment`	Starts a journal segment: empties the place of its generation, writes its header and syncs.
`mqtt_platform_store_copy_begin`, `mqtt_platform_store_copy_record`, `mqtt_platform_store_copy_end`	Write the first copy of a snapshot: its header, its records, then a sync.
`mqtt_platform_store_copy_duplicate`	Writes each other copy from the first, each synced before the next. It can be asked again after it fails.
`mqtt_platform_store_set_aside`	Puts everything the store holds aside, under a number of its own, and starts it again empty. Nothing is deleted: in a folder, the files go to a folder `aside.1`, `aside.2` and so on inside it.
`mqtt_platform_store_copies`, `mqtt_platform_store_erased`, `mqtt_platform_store_capacity`, `mqtt_platform_store_downtime`	What an `mqtt_store_source` and a snapshot need: the copies, the value of a byte never written, the most a place holds, and how long ago the journal was last written. Flash can't tell the last, so on a board intervals don't run while the program is stopped.
`mqtt_platform_parse_durability`	Reads `memory`, `written` or `synced` as a durability.
`mqtt_platform_store_remove`	Removes a store that isn't open, for the tests: a program sets a store aside, and never removes one.

A program that keeps a store on a medium of its own copies the bytes it is given, and needs none of the rest of `base_store.h` (Table 4.10): it is how the library frames, checks and reads the records.

api-reference-en.tex Page 90 November 8, 2026 13:41

Table 4.10 The rest of `base_store.h`

Function	What it does
`mqtt_store_crc16`	The CRC-16 of every header and record: polynomial 0x1021, not reflected, no final XOR, started from `MQTT_STORE_CRC_INIT`.
`mqtt_store_header_encode`, `mqtt_store_header_decode`	Write and check the header that starts each copy and segment. A major version above `MQTT_STORE_FORMAT_MAJOR` (2) is not read.
`mqtt_store_frame_head`, `mqtt_store_frame_check`, `mqtt_store_frame_crc`, `mqtt_store_frame_tail`	Write and check the frame of a record: its kind and length, with a CRC of their own, and the CRC after its content.
`mqtt_store_put_u64`, `mqtt_store_get_u64`	Eight bytes in network byte order.
`mqtt_store_scan_place`	Reads and checks one place of a store, as a restore does first.

4.6 The system

`base_platform.h` declares what the library needs from a system, which each folder of `platform/` gives. A program can use it too, so that it needs nothing of its own for the clock, random bytes or stopping.

4.6.1 `mqtt_platform_time_ms`

Reads the clock, for every `now_ms`.

```
uint64_t mqtt_platform_time_ms(void);
```

See include/base_platform.h

Parameter	Meaning
No parameters.	
Returns	Milliseconds since an arbitrary point. It only goes forward.

api-reference-en.tex Page 91 November 8, 2026 13:41

4.6.2 mqtt_platform_random_bytes

Fills bytes from the system's source of random numbers, which an attacker can't predict: the source for a Server's `random_bytes`.

```
mqtt_status mqtt_platform_random_bytes(uint8_t *data, uint32_t length);
```

See include/base_platform.h

Parameter	Meaning
`data` *out*	Where to put them.
`length` *in*	The number of bytes.
Returns	`MQTT_OK`, or `MQTT_INVALID_STATE` if the system has no such source or it failed, and then `data` means nothing.

4.6.3 mqtt_platform_stop_requested

Finds whether the program has been asked to stop: by a signal, on a POSIX system, or by `mqtt_platform_request_stop` (§4.6.4). On a POSIX system, the first call starts to watch `SIGINT` and `SIGTERM`.

```
bool mqtt_platform_stop_requested(void);
```

See include/base_platform.h

Parameter	Meaning
No parameters.	
Returns	true if it has.

api-reference-en.tex Page 92 November 8, 2026 13:41

4.6.4 `mqtt_platform_request_stop`

Asks the program to stop, as a signal would: for a system with no signals, or a task that ends the others. `mqtt_net_server_run` (§2.6.1) returns soon after.

```
void mqtt_platform_request_stop(void);
```

See include/base_platform.h

Parameter	Meaning
No parameters.	
Returns	Nothing.

The rest of `base_platform.h` is in Table 4.11: the sockets, for a loop of the program's own, as `server_loop.c` uses them, and what the loops of `platform/` use to wait. Its stores are in Section 4.5.

Table 4.11 The other functions of `base_platform.h`

Function	What it does
The program	
`mqtt_platform_time_us`	The clock, in microseconds.
`mqtt_platform_sleep_ms`	Sleeps.
`mqtt_platform_console_write`	Writes text to the diagnostic output.
`mqtt_platform_output_write`	Writes bytes to the normal output.
`mqtt_platform_page_size`	The size of the system's pages, to compare with `MQTT_PAGE_SIZE`, or 0 if the system doesn't say.
`mqtt_platform_parse_u32`	Reads a decimal number from text.
Sockets, which never block	
`mqtt_platform_listen`	Opens a TCP socket that accepts connections.
`mqtt_platform_accept`	Accepts a connection, or gives `MQTT_INCOMPLETE` if none waits.
`mqtt_platform_connect`	Opens a TCP connection, and waits for it.
`mqtt_platform_read`, `mqtt_platform_read_spaces`	Read into one place, or into several in one call. `MQTT_INCOMPLETE` if nothing waits.
`mqtt_platform_write`, `mqtt_platform_write_runs`	Write one run, or several in one call. `MQTT_INCOMPLETE` if nothing was taken.

continues

api-reference-en.tex Page 93 November 8, 2026 13:41

Table 4.11 The other functions of `base_platform.h` (*continued*)

Function	What it does
`mqtt_platform_send_buffer`	Sets the size of a socket's send buffer.
`mqtt_platform_close`	Closes a socket.
`mqtt_platform_wait_socket`	Waits for one socket.
Waiting in tiers, as the loop of a Server does	
`mqtt_platform_wait_start`, `mqtt_platform_wait_add`, `mqtt_platform_wait_interest`, `mqtt_platform_wait_remove`, `mqtt_platform_wait_tier`, `mqtt_platform_wait`	Keep a set of sockets sorted by how busy they are, and wait for those that are due.
`mqtt_platform_wake_handle`, `mqtt_platform_wake_drain`	The socket that a stop makes readable, to end a wait at once.
For tests and benchmarks	
`mqtt_platform_socket_pair`, `mqtt_platform_close_resets`, `mqtt_platform_cpu_us`, `mqtt_platform_interrupt_after_s`, `mqtt_platform_fork`, `mqtt_platform_child_exit`	A system without one says so through what it returns.

4.7 The version of the library

`base_library.h` gives the version of the headers a program is compiled with, and three functions that say what the library it runs with was built as. A program linked with `libterae.a` always has the two the same. A program that loads a shared library, which may have been built elsewhere, for another version or with another page size, checks it when it starts. The version is three numbers, each from 0 to 255:

api-reference-en.tex Page 94 November 8, 2026 13:41

Table 4.12 The version, from `base_library.h`

Macro	Value	Changes when
`MQTT_LIBRARY_VERSION_MAJOR`	1	A program built for the version before may need changing, or building again.
`MQTT_LIBRARY_VERSION_MINOR`	0	The library gains something, and a program built for an earlier minor version still works with it.
`MQTT_LIBRARY_VERSION_PATCH`	5	Nothing a program can see changes.
`MQTT_LIBRARY_VERSION`	`0x010005`	Any of the three: they are a byte each, the major first, so a later version has a larger number.

4.7.1 `mqtt_library_compatible`

Finds whether a program built with headers of a version and a page size can use the library: it has the same major version, a minor version no later than the library's, and the same page size, as the program sizes arrays with `MQTT_PAGE_SIZE` that the library fills. The patch versions don't matter.

```
bool mqtt_library_compatible(uint32_t version, uint32_t page_size);
```

See include/base_library.h

Parameter	Meaning
`version` *in*	The `MQTT_LIBRARY_VERSION` the program was built with.
`page_size` *in*	The `MQTT_PAGE_SIZE` the program was built with.
Returns	true if it can.

A program asks with the numbers of its own headers, before anything else:

```
if (!mqtt_library_compatible(MQTT_LIBRARY_VERSION, MQTT_PAGE_SIZE)) {
    mqtt_platform_console_write("libterae: another version or page size\n");
    return 1;
}
```

api-reference-en.tex Page 95 November 8, 2026 13:41

4.7.2 mqtt_library_version

Finds the version the library was built as, to report it.

```
uint32_t mqtt_library_version(void);
```

See include/base_library.h

Parameter	Meaning
No parameters.	
Returns	Its `MQTT_LIBRARY_VERSION`.

4.7.3 mqtt_library_page_size

Finds the page size the library was built with.

```
uint32_t mqtt_library_page_size(void);
```

See include/base_library.h

Parameter	Meaning
No parameters.	
Returns	Its `MQTT_PAGE_SIZE`.

5

Examples

These programs were built against the library as they are here, with `-std=c99 -Wall -Wextra -Wpedantic -Werror`, and run together: each Server, with one Client subscribed and one publishing three messages, which the first printed. `server_store.c` was also stopped and started again twice, and a message that waited for a Client away all that time reached it. Build one with its set-up file:

```
cc -std=c99 -Iinclude -DMQTT_PAGE_SIZE=16384U server_platform.c server_setup.c \
   build/libterae.a -o server_platform
cc -std=c99 -Iinclude -DMQTT_PAGE_SIZE=16384U server_store.c server_setup.c \
   build/libterae.a -o server_store
cc -std=c99 -Iinclude -DMQTT_PAGE_SIZE=16384U client_subscribe.c client_setup.c \
   build/libterae.a -o client_subscribe
```

5.1 Servers

`server_setup.c` gives a Server its memory and configuration, for up to 64 connections, and `server_setup.h` declares it. The three Servers below use it. Given a store, it sets the configuration to keep the server's Session state there.

server_setup.h

```
/* What the example Servers share: their memory and configuration. */
#ifndef SERVER_SETUP_H
#define SERVER_SETUP_H

#include <stdint.h>

#include "base_server.h"

/* The number of connections, and so of entries a network layer needs. */
#define CONNECTIONS 64U
/* The messages and deliveries the server holds, and the entries a restore
 * needs to find them by their numbers in a store: both together. */
#define MESSAGES        512U
#define DELIVERIES      1024U
```

api-reference-en.tex Page 98 November 8, 2026 13:41


```
#define RESTORE_ENTRIES (MESSAGES + DELIVERIES)

/* Give the server its memory and configuration, and set it up. With a store
 * from mqtt_platform_store_open, it keeps its Session state there; with
 * MQTT_STORE_NONE, nothing outlasts a stop. */
mqtt_status server_setup(mqtt_server *server, int32_t store);

#endif /* SERVER_SETUP_H */
```

server_setup.c

```
/* The memory and the configuration of the example Servers. */
#include <stdint.h>

#include "base_net.h"
#include "base_platform.h"
#include "base_server.h"
#include "server_setup.h"

/* How much of everything the server gets: these numbers are its limits.
 * CONNECTIONS, MESSAGES and DELIVERIES are in server_setup.h, as the network
 * layer and a restore need them too. */
#define BLOCK_SIZE    1024U  /* bytes in a block of the pool */
#define BLOCKS        512U   /* more than pool_borrow_reserve + hard quota */
#define SCRATCH       16384U
#define SESSIONS      64U
#define SUBSCRIPTIONS 256U
#define GROUPS        32U
#define NODES         256U
#define TEXT_PAGES    4U
#define ARENA_PAGES   64U
#define ALIASES       16U    /* the topic_alias_maximum of the configuration */

/* Pages of every arena, each count rounded up to whole pages. */
#define PAGES(count, per_page) (((count) + (per_page) - 1U) / (per_page))
#define META_PAGES                                                    \
    (PAGES(CONNECTIONS, MQTT_SERVER_PAGE_CONNECTIONS) +               \
     PAGES(SESSIONS, MQTT_SERVER_PAGE_SESSIONS) +                     \
     PAGES(DELIVERIES, MQTT_SERVER_PAGE_DELIVERIES) +                 \
     PAGES(MESSAGES, MQTT_SERVER_PAGE_MESSAGES) +                     \
     PAGES(SUBSCRIPTIONS, MQTT_SERVER_PAGE_SUBSCRIPTIONS) +           \
     PAGES(GROUPS, MQTT_SERVER_PAGE_GROUPS) +                         \
     PAGES(NODES, MQTT_TREE_PAGE_NODES) + TEXT_PAGES + ARENA_PAGES)

static mqtt_connection connections[CONNECTIONS];
static uint8_t pool_bytes[BLOCKS * BLOCK_SIZE];
static uint32_t pool_next[BLOCKS];
static uint8_t scratch[SCRATCH];
static mqtt_topic_alias aliases[CONNECTIONS * ALIASES];
static mqtt_session sessions[SESSIONS];
static uint32_t session_table[SESSIONS];
static mqtt_delivery deliveries[DELIVERIES];
static mqtt_message messages[MESSAGES];
static mqtt_subscription subscriptions[SUBSCRIPTIONS];
static mqtt_share_group groups[GROUPS];
static mqtt_tree_node nodes[NODES];
static mqtt_tree_branch branches[NODES];
static uint8_t topic_text[TEXT_PAGES * MQTT_PAGE_SIZE];
static uint8_t arena[ARENA_PAGES * MQTT_PAGE_SIZE];
static uint32_t page_meta[META_PAGES * MQTT_PAGE_META_WORDS];

/* Assigned Client Identifiers made from the system's random bytes. */
static mqtt_status random_bytes(uint32_t tag, uint8_t *data, uint32_t length)
```

api-reference-en.tex Page 99 November 8, 2026 13:41


```
{
    (void)tag;
    return mqtt_platform_random_bytes(data, length);
}

mqtt_status server_setup(mqtt_server *server, int32_t store)
{
    mqtt_server_config config;
    mqtt_server_storage storage;

    mqtt_server_config_default(&config);
    config.random_bytes = &random_bytes;
    config.topic_alias_maximum = ALIASES;
    if (store != MQTT_STORE_NONE) {
        /* Sessions and messages outlast a loss of power, and every callback
         * is given the store as its tag. */
        mqtt_net_server_store_config(&config, store);
    }

    storage.connections = connections;
    storage.connection_count = CONNECTIONS;
    storage.pool_bytes = pool_bytes;
    storage.pool_next = pool_next;
    storage.pool_block_size = BLOCK_SIZE;
    storage.pool_block_count = BLOCKS;
    storage.receive_scratch = scratch;
    storage.receive_scratch_size = SCRATCH;
    storage.aliases = aliases;
    storage.sessions = sessions;
    storage.session_count = SESSIONS;
    storage.session_table = session_table;
    storage.session_table_count = SESSIONS;
    storage.deliveries = deliveries;
    storage.delivery_count = DELIVERIES;
    storage.messages = messages;
    storage.message_count = MESSAGES;
    storage.subscriptions = subscriptions;
    storage.subscription_count = SUBSCRIPTIONS;
    storage.groups = groups;
    storage.group_count = GROUPS;
    storage.nodes = nodes;
    storage.node_count = NODES;
    storage.branches = branches;
    storage.branch_count = NODES;
    storage.topic_text = topic_text;
    storage.topic_text_pages = TEXT_PAGES;
    storage.arena_bytes = arena;
    storage.arena_pages = ARENA_PAGES;
    storage.page_meta = page_meta;
    storage.page_meta_words = META_PAGES * MQTT_PAGE_META_WORDS;

    return mqtt_server_init(server, &config, &storage);
}
```

`server_platform.c` runs it with `platform/`: the whole program is the options and one call.

api-reference-en.tex Page 100 November 8, 2026 13:41

server_platform.c

```
/* A Server on TCP port 1883, run by platform/ until Ctrl-C. */
#include <stdio.h>

#include "base_net.h"
#include "base_platform.h"
#include "base_server.h"
#include "server_setup.h"

static mqtt_server server;
static mqtt_net_connection network[CONNECTIONS];

int main(void)
{
    mqtt_net_server_options options;
    mqtt_status status = server_setup(&server, MQTT_STORE_NONE);

    if (status == MQTT_OK) {
        options.host = NULL;             /* every local address */
        options.port = "1883";           /* MQTT over TCP */
        options.tls_port = NULL;         /* no TLS */
        options.websocket_port = NULL;   /* no WebSocket */
        options.certificate_file = NULL;
        options.key_file = NULL;
        options.key_password = NULL;
        options.ca_file = NULL;
        options.tier = NULL;             /* the default tiers */
        options.send_buffer = 0U;        /* the system's send buffer */
        options.store = MQTT_STORE_NONE; /* nothing outlasts a stop */
        status = mqtt_net_server_run(&server, network, &options);
    }
    if (status != MQTT_OK) {
        fprintf(stderr, "the server stopped: status %d\n", (int)status);
    }
    return (status == MQTT_OK) ? 0 : 1;
}
```

`server_store.c` is the same with a store, in the folder `terae-store`: stopped and started again, it goes on with the Sessions it had (Section 2.9).

server_store.c

```
/* A Server on TCP port 1883, run by platform/ until Ctrl-C, that keeps its
 * Sessions, their messages and the retained messages in the folder
 * terae-store: started again, it goes on where it stopped. */
#include <stdio.h>

#include "base_net.h"
#include "base_platform.h"
#include "base_server.h"
#include "server_setup.h"

static mqtt_server server;
static mqtt_net_connection network[CONNECTIONS];
/* Room for a restore to find each message and delivery by its number. */
static mqtt_server_restore_entry map[RESTORE_ENTRIES];

int main(void)
{
    int32_t store = MQTT_STORE_NONE;
    mqtt_store_fault fault;
```

api-reference-en.tex Page 101 November 8, 2026 13:41


```
    mqtt_net_server_options options;
    /* The folder is made if it isn't there, with two copies of the snapshot,
     * so that one is whole whenever the other is being written. */
    mqtt_status status = mqtt_platform_store_open("terae-store", 2U, &store);

    if (status == MQTT_OK) {
        status = server_setup(&server, store);
    }
    if (status == MQTT_OK) {
        /* What the store holds comes back. One that is damaged is set aside,
         * with a line on the console, and the server starts empty. */
        status = mqtt_net_server_restore(&server, store, map, RESTORE_ENTRIES,
                                         &fault);
    }
    if (status == MQTT_OK) {
        options.host = NULL;
        options.port = "1883";
        options.tls_port = NULL;
        options.websocket_port = NULL;
        options.certificate_file = NULL;
        options.key_file = NULL;
        options.key_password = NULL;
        options.ca_file = NULL;
        options.tier = NULL;
        options.send_buffer = 0U;
        options.store = store; /* written, synced and copied as it runs */
        status = mqtt_net_server_run(&server, network, &options);
    }
    mqtt_platform_store_close(store); /* nothing, if it never opened */
    if (status != MQTT_OK) {
        fprintf(stderr, "the server stopped: status %d\n", (int)status);
    }
    return (status == MQTT_OK) ? 0 : 1;
}
```

`server_loop.c` runs it in a loop of its own, with the sockets of `base_platform.h`, to show the calls `mqtt_net_server_run` (§2.6.1) makes.

server_loop.c

```
/* A Server on TCP port 1883 in a loop of its own, until Ctrl-C. It looks at
 * every socket every few milliseconds, which keeps it short: a real loop waits
 * for the sockets that are ready. */
#include <stdint.h>
#include <stdio.h>

#include "base_platform.h"
#include "base_server.h"
#include "server_setup.h"

#define ROUND_MS 5U /* the longest it sleeps between two rounds */

static mqtt_server server;
static int32_t sockets[CONNECTIONS]; /* by the server's connection index */

static void close_connection(uint32_t c)
{
    mqtt_platform_close(sockets[c]);
    sockets[c] = MQTT_SOCKET_NONE;
    mqtt_server_connection_closed(&server, c, mqtt_platform_time_ms());
}

/* Move what can be moved on one connection: in, then out. */
```

api-reference-en.tex Page 102 November 8, 2026 13:41


```
static void serve(uint32_t c)
{
    mqtt_status status = MQTT_OK;

    if (mqtt_server_receive_wanted(&server, c)) {
        mqtt_space spaces[MQTT_PLATFORM_READ_SPACES];
        const uint32_t count = mqtt_server_receive_space(
            &server, c, spaces, MQTT_PLATFORM_READ_SPACES);
        uint32_t got = 0U;

        if (count > 0U) {
            status = mqtt_platform_read_spaces(sockets[c], spaces, count, &got);
            if (status != MQTT_OK) {
                got = 0U; /* MQTT_INCOMPLETE: nothing has come */
            }
            /* Always after receive_space, even with nothing. */
            (void)mqtt_server_receive_commit(&server, c, got,
                                             mqtt_platform_time_ms());
        }
    }
    if (status != MQTT_CONNECTION_CLOSED) {
        mqtt_run runs[MQTT_PLATFORM_WRITE_RUNS];
        const uint32_t count = mqtt_server_transmit_view(
            &server, c, runs, MQTT_PLATFORM_WRITE_RUNS);
        uint32_t taken = 0U;

        if (count > 0U) {
            status = mqtt_platform_write_runs(sockets[c], runs, count, &taken);
            if (status == MQTT_OK) {
                mqtt_server_transmit_consume(&server, c, taken);
            }
        }
    }
    if ((status == MQTT_CONNECTION_CLOSED) ||
        mqtt_server_should_close(&server, c)) {
        close_connection(c);
    }
}

int main(void)
{
    int32_t listener = MQTT_SOCKET_NONE;
    int32_t handle = MQTT_SOCKET_NONE;
    uint32_t c = 0U;

    for (c = 0U; c < CONNECTIONS; c++) {
        sockets[c] = MQTT_SOCKET_NONE;
    }
    if ((server_setup(&server, MQTT_STORE_NONE) != MQTT_OK) ||
        (mqtt_platform_listen(NULL, "1883", &listener) != MQTT_OK)) {
        fprintf(stderr, "the server could not start\n");
        return 1;
    }
    while (!mqtt_platform_stop_requested()) {
        /* The timers, which also say how long the loop may sleep. */
        uint32_t wait_ms = mqtt_server_tick(&server, mqtt_platform_time_ms());

        /* New connections. */
        while (mqtt_platform_accept(listener, &handle) == MQTT_OK) {
            if ((mqtt_server_connection_open(&server, mqtt_platform_time_ms(),
                                             &c) == MQTT_OK) &&
                (c < CONNECTIONS)) {
                sockets[c] = handle;
            } else {
                mqtt_platform_close(handle); /* no room for another */
```

api-reference-en.tex Page 103 November 8, 2026 13:41


```
            }
        }
        /* Every open connection. */
        for (c = 0U; c < CONNECTIONS; c++) {
            if (sockets[c] != MQTT_SOCKET_NONE) {
                serve(c);
            }
        }
        mqtt_platform_sleep_ms((wait_ms < ROUND_MS) ? wait_ms : ROUND_MS);
    }
    for (c = 0U; c < CONNECTIONS; c++) {
        if (sockets[c] != MQTT_SOCKET_NONE) {
            close_connection(c);
        }
    }
    mqtt_platform_close(listener);
    return 0;
}
```

5.2 Clients

`client_setup.c` gives a Client its memory, and `client_setup.h` declares it.

client_setup.h

```
/* What the example Clients share: their memory. */
#ifndef CLIENT_SETUP_H
#define CLIENT_SETUP_H

#include "base_client.h"

/* Give the client its memory, with on_event to be told what happens. */
mqtt_status client_setup(mqtt_client *client,
                         mqtt_client_event_function on_event);

#endif /* CLIENT_SETUP_H */
```

client_setup.c

```
/* The memory of the example Clients. */
#include <stdint.h>

#include "base_client.h"
#include "client_setup.h"

#define RECEIVE_BYTES 65536U  /* the largest packet it can take */
#define BLOCK_SIZE    64U
#define BLOCKS        1024U   /* 64 KiB of bytes waiting to be sent */
#define MESSAGES      64U
#define INCOMING      64U     /* its Receive Maximum */
#define PENDING       8U
#define ALIASES       16U     /* its Topic Alias Maximum */
#define ARENA_UNITS   1024U   /* 64 KiB for the packets it holds */

static uint8_t receive_bytes[RECEIVE_BYTES];
static uint8_t pool_bytes[BLOCKS * BLOCK_SIZE];
static uint32_t pool_next[BLOCKS];
static mqtt_client_message messages[MESSAGES];
```

api-reference-en.tex Page 104 November 8, 2026 13:41


```
static uint16_t incoming[INCOMING];
static mqtt_client_pending pending[PENDING];
static mqtt_client_alias aliases[ALIASES];
static uint8_t arena_bytes[ARENA_UNITS * MQTT_ARENA_UNIT];
static uint8_t arena_states[ARENA_UNITS];

mqtt_status client_setup(mqtt_client *client,
                         mqtt_client_event_function on_event)
{
    mqtt_client_storage storage;

    storage.on_event = on_event;
    storage.user_tag = 0U;
    storage.receive_bytes = receive_bytes;
    storage.receive_size = RECEIVE_BYTES;
    storage.pool_bytes = pool_bytes;
    storage.pool_next = pool_next;
    storage.pool_block_size = BLOCK_SIZE;
    storage.pool_block_count = BLOCKS;
    storage.messages = messages;
    storage.message_count = MESSAGES;
    storage.incoming = incoming;
    storage.incoming_count = INCOMING;
    storage.pending = pending;
    storage.pending_count = PENDING;
    storage.aliases = aliases;
    storage.alias_count = ALIASES;
    storage.arena_bytes = arena_bytes;
    storage.arena_states = arena_states;
    storage.arena_units = ARENA_UNITS;
    return mqtt_client_init(client, &storage);
}
```

`client_subscribe.c` subscribes to `a/b` with MQTT 5.0, prints the first three messages that come, and disconnects.

client_subscribe.c

```
/* A Client that subscribes to a/b and prints the first three messages. */
#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>

#include "base_client.h"
#include "base_net.h"
#include "base_platform.h"
#include "client_setup.h"
#include "v5_rc.h"

#define STEP_MS 100U

static mqtt_client client;
static bool accepted = false;   /* a CONNACK accepted the connection */
static bool refused = false;    /* the Server said no */
static bool subscribed = false; /* the SUBACK came */
static uint32_t received = 0U;

/* Everything the Server sends comes here, during mqtt_net_client_step. */
static void on_event(uint32_t tag, const mqtt_client_event *event)
{
    (void)tag;
    switch (event->type) {
    case MQTT_CLIENT_EVENT_CONNACK:
```

api-reference-en.tex Page 105 November 8, 2026 13:41


```
        accepted = (event->connack->reason_code == 0U);
        refused = !accepted;
        break;
    case MQTT_CLIENT_EVENT_SUBACK:
        subscribed = true;
        refused = (event->subscribed->reason_codes.length == 0U) ||
                  (event->subscribed->reason_codes.data[0] >=
                   MQTT5_RC_FAILURE_MIN);
        break;
    case MQTT_CLIENT_EVENT_MESSAGE:
        printf("%.*s %.*s\n", (int)event->publish->topic_name.length,
               (const char *)event->publish->topic_name.data,
               (int)event->publish->payload.length,
               (const char *)event->publish->payload.data);
        received += 1U;
        break;
    default:
        break;
    }
}

/* Step the connection until *flag is set, it is refused, or it ends. */
static bool wait_for(mqtt_net_client *network, const bool *flag)
{
    mqtt_status status = MQTT_OK;

    while (!*flag && !refused && (status == MQTT_OK) &&
           !mqtt_platform_stop_requested()) {
        status = mqtt_net_client_step(network, STEP_MS);
    }
    return *flag && !refused;
}

int main(void)
{
    mqtt_net_client_tls tls;
    mqtt_net_client network;
    mqtt_connect connect;
    mqtt_client_subscription entry;
    uint16_t packet_identifier;
    bool done = false;

    if (client_setup(&client, &on_event) != MQTT_OK) {
        return 1;
    }

    tls.enabled = false;
    tls.insecure = false;
    tls.ca_file = NULL;
    tls.server_name = NULL;
    tls.certificate_file = NULL;
    tls.key_file = NULL;
    tls.key_password = NULL;
    if (mqtt_net_client_open(&network, &client, "localhost", "1883", &tls) !=
        MQTT_OK) {
        return 1;
    }

    mqtt_connect_clear(&connect);
    connect.version = MQTT_VERSION_5;
    connect.clean_start = true;
    connect.keep_alive = 60U;
    connect.client_identifier = mqtt_utf8_from_text("reader");
    if ((mqtt_client_connect(&client, &connect, mqtt_platform_time_ms()) ==
         MQTT_OK) &&
```

api-reference-en.tex Page 106 November 8, 2026 13:41


```
        wait_for(&network, &accepted)) {
        entry.topic_filter = mqtt_utf8_from_text("a/b");
        entry.options = 1U; /* QoS 1, and no other option */
        if ((mqtt_client_subscribe(&client, &entry, 1U, NULL,
                                   &packet_identifier) == MQTT_OK) &&
            wait_for(&network, &subscribed)) {
            fprintf(stderr, "subscribed\n");
            while ((received < 3U) && !mqtt_platform_stop_requested() &&
                   (mqtt_net_client_step(&network, STEP_MS) == MQTT_OK)) {
            }
            done = (received == 3U);
        }
    }
    if (mqtt_client_disconnect(&client, NULL) == MQTT_OK) {
        while (mqtt_net_client_step(&network, STEP_MS) == MQTT_OK) {
        }
    }
    mqtt_net_client_close(&network);
    return done ? 0 : 1;
}
```

`client_publish.c` publishes three messages to `a/b` at QoS 1, with a Content Type and a User Property, and waits for each to be acknowledged. Its CONNECT asks for a Session that outlives the connection by five minutes.

client_publish.c

```
/* A Client that publishes three messages to a/b at QoS 1, with Properties. */
#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>

#include "base_client.h"
#include "base_net.h"
#include "base_platform.h"
#include "client_setup.h"
#include "v5_prop.h"

#define STEP_MS 100U
#define COUNT   3U

static mqtt_client client;
static bool accepted = false;  /* a CONNACK accepted the connection */
static bool refused = false;   /* the Server said no */
static uint32_t answered = 0U; /* messages whose flow has ended */

static void on_event(uint32_t tag, const mqtt_client_event *event)
{
    (void)tag;
    if (event->type == MQTT_CLIENT_EVENT_CONNACK) {
        accepted = (event->connack->reason_code == 0U);
        refused = !accepted;
    } else if (event->type == MQTT_CLIENT_EVENT_PUBLISHED) {
        /* The PUBACK of a QoS 1 message: 0x00, or 0x10 if nobody listens. */
        fprintf(stderr, "packet %u answered with 0x%02x\n",
                (unsigned)event->response->packet_identifier,
                (unsigned)event->response->reason_code);
        answered += 1U;
    } else {
        /* Nothing else matters here. */
    }
```

api-reference-en.tex Page 107 November 8, 2026 13:41


```
}

int main(void)
{
    static const mqtt5_utf8_pair origin = {{6U, (const uint8_t *)"source"},
                                           {7U, (const uint8_t *)"example"}};
    mqtt_net_client_tls tls;
    mqtt_net_client network;
    mqtt_connect connect;
    mqtt_publish message;
    char text[32];
    uint32_t sent = 0U;
    mqtt_status status = MQTT_OK;

    if (client_setup(&client, &on_event) != MQTT_OK) {
        return 1;
    }
    memset(&tls, 0, sizeof(tls)); /* no TLS */
    if (mqtt_net_client_open(&network, &client, "localhost", "1883", &tls) !=
        MQTT_OK) {
        return 1;
    }

    mqtt_connect_clear(&connect);
    connect.version = MQTT_VERSION_5;
    connect.clean_start = true;
    connect.keep_alive = 60U;
    connect.client_identifier = mqtt_utf8_from_text("writer");
    /* A Session that outlives the connection by five minutes. */
    connect.properties.session_expiry_interval = 300U;
    mqtt5_properties_mark(&connect.properties,
                          MQTT5_PROP_SESSION_EXPIRY_INTERVAL);
    status = mqtt_client_connect(&client, &connect, mqtt_platform_time_ms());
    while ((status == MQTT_OK) && !accepted && !refused) {
        status = mqtt_net_client_step(&network, STEP_MS);
    }

    while ((status == MQTT_OK) && accepted && (answered < COUNT) &&
           !mqtt_platform_stop_requested()) {
        if (sent < COUNT) {
            memset(&message, 0, sizeof(message)); /* no Properties */
            snprintf(text, sizeof(text), "hello %u", (unsigned)(sent + 1U));
            message.qos = 1U;
            message.topic_name = mqtt_utf8_from_text("a/b");
            message.payload.data = (const uint8_t *)text;
            message.payload.length = (uint32_t)strlen(text);
            message.properties.content_type =
                mqtt_utf8_from_text("text/plain");
            mqtt5_properties_mark(&message.properties,
                                  MQTT5_PROP_CONTENT_TYPE);
            message.properties.user_pairs = &origin;
            message.properties.user_pair_count = 1U;
            /* The message is copied: text can be reused at once. */
            if (mqtt_client_publish(&client, &message,
                                    mqtt_platform_time_ms()) == MQTT_OK) {
                sent += 1U;
            }
        }
        status = mqtt_net_client_step(&network, (sent < COUNT) ? 0U : STEP_MS);
    }

    if (mqtt_client_disconnect(&client, NULL) == MQTT_OK) {
        while (mqtt_net_client_step(&network, STEP_MS) == MQTT_OK) {
        }
    }
```

api-reference-en.tex Page 108 November 8, 2026 13:41


```
    mqtt_net_client_close(&network);
    return (answered == COUNT) ? 0 : 1;
}
```



A

Every Function

The functions of this document by name, with their header and the section that describes them.

Table A.1 Every function, by name

Function	Header	§
mqtt5_properties_clear	v5_properties.h	4.2.1
mqtt5_properties_has	v5_properties.h	4.2.3
mqtt5_properties_mark	v5_properties.h	4.2.2
mqtt5_property_next	v5_properties.h	4.2.4
mqtt5_reason_code	base_packet.h	4.3.3
mqtt_client_assume_session	base_client.h	3.5.2
mqtt_client_auth	base_client.h	3.6.2
mqtt_client_connect	base_client.h	3.6.1
mqtt_client_connection_lost	base_client.h	3.6.5
mqtt_client_disconnect	base_client.h	3.6.3
mqtt_client_init	base_client.h	3.5.1
mqtt_client_publish	base_client.h	3.7.1
mqtt_client_receive	base_client.h	3.9.4
mqtt_client_receive_capacity	base_client.h	3.9.1
mqtt_client_receive_commit	base_client.h	3.9.3
mqtt_client_receive_space	base_client.h	3.9.2
mqtt_client_restore	base_client.h	3.11.1
mqtt_client_should_close	base_client.h	3.6.4

continues

api-reference-en.tex Page 110 November 8, 2026 13:41

Table A.1 Every function, by name (*continued*)

Function	Header	§
mqtt_client_snapshot_abort	base_client.h	3.11.10
mqtt_client_snapshot_begin	base_client.h	3.11.7
mqtt_client_snapshot_done	base_client.h	3.11.9
mqtt_client_snapshot_due	base_client.h	3.11.6
mqtt_client_snapshot_next	base_client.h	3.11.8
mqtt_client_store_commit	base_client.h	3.11.2
mqtt_client_store_failed	base_client.h	3.11.4
mqtt_client_store_identifier	base_client.h	3.10.4
mqtt_client_store_init	base_client.h	3.10.1
mqtt_client_store_open	base_client.h	3.11.5
mqtt_client_stored	base_client.h	3.11.3
mqtt_client_subscribe	base_client.h	3.7.2
mqtt_client_tick	base_client.h	3.9.7
mqtt_client_transmit_consume	base_client.h	3.9.6
mqtt_client_transmit_view	base_client.h	3.9.5
mqtt_client_unsubscribe	base_client.h	3.7.3
mqtt_connect_clear	base_packet.h	4.3.2
mqtt_library_compatible	base_library.h	4.7.1
mqtt_library_page_size	base_library.h	4.7.3
mqtt_library_version	base_library.h	4.7.2
mqtt_net_client_close	base_net.h	3.8.3
mqtt_net_client_open	base_net.h	3.8.1
mqtt_net_client_restore	base_net.h	3.10.3
mqtt_net_client_step	base_net.h	3.8.2
mqtt_net_client_store_config	base_net.h	3.10.2
mqtt_net_server_counters	base_net.h	2.6.2
mqtt_net_server_restore	base_net.h	2.9.2
mqtt_net_server_run	base_net.h	2.6.1
mqtt_net_server_store_config	base_net.h	2.9.1
mqtt_platform_random_bytes	base_platform.h	4.6.2
mqtt_platform_request_stop	base_platform.h	4.6.4

continues

api-reference-en.tex Page 111 November 8, 2026 13:41

Table A.1 Every function, by name (*continued*)

Function	Header	§
mqtt_platform_stop_requested	base_platform.h	4.6.3
mqtt_platform_store_close	base_platform.h	4.5.3
mqtt_platform_store_open	base_platform.h	4.5.2
mqtt_platform_time_ms	base_platform.h	4.6.1
mqtt_reason_clear	base_packet.h	4.3.1
mqtt_server_accepted	base_server.h	2.7.14
mqtt_server_config_default	base_server.h	2.5.1
mqtt_server_connection_closed	base_server.h	2.7.2
mqtt_server_connection_open	base_server.h	2.7.1
mqtt_server_disconnect	base_server.h	2.7.17
mqtt_server_idle	base_server.h	2.7.16
mqtt_server_init	base_server.h	2.5.3
mqtt_server_keep_alive	base_server.h	2.7.15
mqtt_server_meta_words	base_server.h	2.5.2
mqtt_server_next_pending	base_server.h	2.7.11
mqtt_server_pool_audit	base_server.h	2.11.1
mqtt_server_publish	base_server.h	2.8.1
mqtt_server_receive	base_server.h	2.7.7
mqtt_server_receive_capacity	base_server.h	2.7.4
mqtt_server_receive_commit	base_server.h	2.7.6
mqtt_server_receive_space	base_server.h	2.7.5
mqtt_server_receive_wanted	base_server.h	2.7.3
mqtt_server_restore	base_server.h	2.10.1
mqtt_server_retained_next	base_server.h	2.8.2
mqtt_server_retained_restore	base_server.h	2.8.3
mqtt_server_should_close	base_server.h	2.7.10
mqtt_server_snapshot_abort	base_server.h	2.10.10
mqtt_server_snapshot_begin	base_server.h	2.10.7
mqtt_server_snapshot_done	base_server.h	2.10.9
mqtt_server_snapshot_due	base_server.h	2.10.6
mqtt_server_snapshot_next	base_server.h	2.10.8

continues

api-reference-en.tex Page 112 November 8, 2026 13:41

Table A.1 Every function, by name (*continued*)

Function	Header	§
mqtt_server_store_commit	base_server.h	2.10.2
mqtt_server_store_failed	base_server.h	2.10.4
mqtt_server_store_open	base_server.h	2.10.5
mqtt_server_stored	base_server.h	2.10.3
mqtt_server_take_activity	base_server.h	2.7.13
mqtt_server_tick	base_server.h	2.7.12
mqtt_server_transmit_consume	base_server.h	2.7.9
mqtt_server_transmit_view	base_server.h	2.7.8
mqtt_store_fault_name	base_store.h	4.5.1
mqtt_tls_available	base_tls.h	4.4.1
mqtt_utf8_from_text	base_wire.h	4.1.1